

The Scaling Laws of Skills in LLM Agent Systems

§ Evolver AI Team

github.com/evolver-ai/skill-laws

evolver.co/en/research

ABSTRACT

As agent systems scale, skills accumulate into large reusable libraries, yet their scaling laws remain poorly understood. Across 15 frontier LLMs, 1,141 real-world skills, and over 3M routing or execution decisions, we identify two coupled laws. **Routing law**: single-step routing accuracy decays logarithmically with library size ($R^2 > 0.97$ for all models), with errors progressing from local skill competition to cross-family drift and capture by overly general “black-hole skills”. **Execution law**: before state realization, joint routing is approximately multiplicative, whereas correct execution can improve difficult downstream decisions by about 4×. A single parameter, the routing logarithmic decay slope b , couples the two laws: routing-side fits predict execution-side rescue across models, showing that the same library property controls both pre-execution collapse and downstream recoverability. The laws are actionable: law-guided optimization raises held-out routing accuracy from 71.3% to 91.7%, reduces hijack from 22.4% to 4.1%, and transfers directionally to downstream CLAWBENCH and CLAWMARK execution settings, improving mean pass rate from 49.3% to 61.6% on CLAWBENCH and from 28.4% to 34.5% on CLAWMARK. These results show that agent performance depends not only on model capability, but also on the structure, granularity, and exposure policy of the skill library.



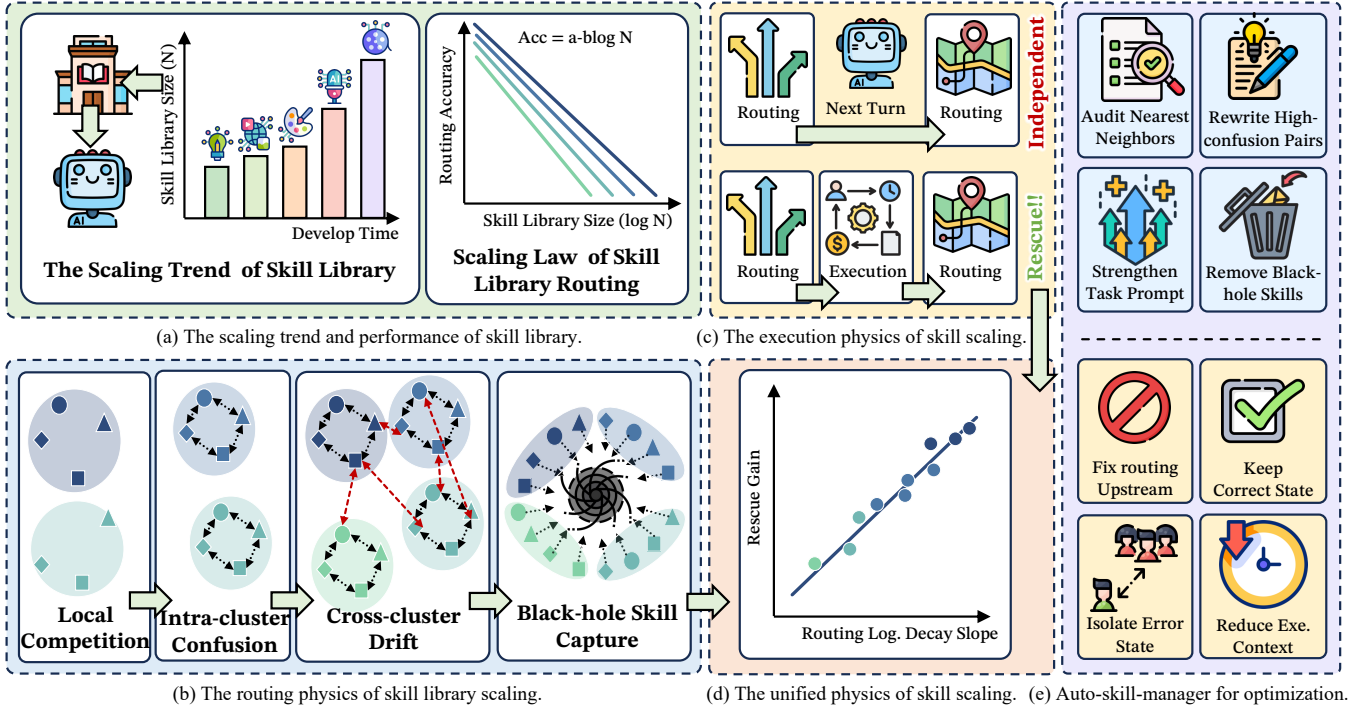


Figure 1: Skill-library scaling laws and optimization implications. (a) Reusable skills accumulate into larger exposed libraries. Routing accuracy decays approximately linearly with $\log N$, with model-dependent slope b . (b) Errors progress from local competition to cross-family drift and finally capture by overly general “black-hole” skills. (c) Execution separates pre-state compounding from post-state rescue. (d) The same library property links routing collapse and execution rescue: b predicts rescue across models without refitting. (e) The laws suggest concrete optimization actions: audit neighbors, rewrite confused pairs, strengthen prompts, and apply additional manager actions.

1. Introduction

As agent systems develop, skills increasingly accumulate into reusable libraries [1, 2, 3]. Early action-selection systems [4, 5, 6] and benchmarks [7, 8] showed that language models can select from large sets of callable interfaces. More recent skill-centric work [9, 10] further treats reusable skills as a central abstraction for building capable agents [11, 12, 13, 14]. Together, these systems suggest a natural scaling premise: expanding a skill library should enlarge the space of behaviors available to an agent, while remaining failures can be attributed to limited model capability or imperfect routing.

This shifts the central question from “can the model use a skill?” to “how does a growing skill library reshape the agent’s routing and execution dynamics?” Existing benchmarks typically vary tasks, models, or action formats, but keep the library as background infrastructure. We instead make library size and structure the object of study. The resulting failure mode is not ordinary hallucination. In our plain-text routing protocol, vague prompts can yield 0% hallucinated skill names while task-level routing accuracy falls to 18.3% and in-library hijack exceeds 80%. The model stays inside the library, but lands in the wrong part of it.

To address this problem, we characterize this behavior through empirical laws of skill libraries. Across 15 frontier LLMs, 1,141 real-world skills, and over 3M routing or execution decisions, we find two coupled empirical laws. The *routing law* governs the pre-execution regime: single-step accuracy follows $Acc(N) = a - b \ln N$ with $R^2 > 0.97$ for every model, while errors progress from local competition among neighboring skills to cross-family drift and finally capture by overly general “black-hole” skills. The *execution law* starts after state is realized. Before any artifact exists, multi-skill routing composes approximately multiplicatively; after correct execution, concrete state

can rescue difficult downstream decisions, producing about a 4× gain on hard pairs. Thus execution does not simply add another chance to fail. It can either propagate bad state or provide evidence that disambiguates the next route. The two laws are connected by a single interpretable parameter: the routing decay slope b . A larger b means that accuracy collapses sooner as the library grows, reducing the probability that correct upstream state is available for downstream rescue. Conversely, when routing-side fits are combined with an independently measured rescue coefficient, b predicts execution-side rescue across models without refitting ($\rho=0.74$, $p<0.001$). This coupling is the central result of the paper: the same library property that controls pre-execution routing collapse also controls whether execution can provide useful downstream state. It turns the laws from retrospective descriptions into design constraints: reduce local competition, sharpen skill boundaries, remove abstract attractors, and expose execution state when it is likely to help.

Guided by these laws, we instantiate a law-guided auto skill manager at the law-predicted transition point for median- b agents. The manager applies four library-side actions: nearest-neighbor auditing, description-boundary rewriting, abstract-skill removal, and prompt anchoring. On 1,600 held-out tasks, it improves routing accuracy from 71.3% to 91.7% and reduces the hijack rate from 22.4% to 4.1%. Factorial ablations identify boundary rewriting (+12.8%) and abstract-skill removal (+4.9%) as the main contributors, consistent with the predicted failure stages. The same design transfers directionally to downstream CLAWBENCH/CLAWMARK execution settings: mean pass rate improves from 49.3% to 61.6% on CLAWBENCH, and from 28.4% to 34.5% on CLAWMARK.

Our contributions are:

- **Two-stage laws of skill libraries.** We establish coupled routing and execution laws that characterize how large skill libraries fail before state is realized and recover after correct execution.
- **A unified law structure for skill libraries.** We show that the routing decay slope b links library-scale routing collapse to execution-side rescue across models without refitting.
- **Guidance for optimizing skill-based agent systems.** We turn the laws into library optimization methods for exposure control, boundary rewriting, attractor removal, and execution-state use, with downstream validation.

2. Related Work

Our work sits at the intersection of skill routing, execution-aware agent systems, and scaling laws. Existing work studies how agents select or use external capabilities, but not the coupled routing and execution laws that appear as the skill library itself scales.

Routing over external capabilities has been studied through callable-interface systems, agent benchmarks, and skill-centric agents. Toolformer [4], ToolLLM [5], ToolAlpaca [15], Gorilla [6], API-Bank [7], TaskBench [8], OpenAgents [16], AgentBench [17], and recent surveys and skill-focused work [1, 2, 3, 14, 18, 9, 10] establish large external action spaces as a core agent interface. Closest to our routing setting, ComplexFuncBench [19] studies multi-step function selection, BiasBusters [20] studies systematic selection bias, and xRouter, EvoRoute, AutoTool, and MetaToolAgent optimize routing or tool choice [21, 22, 23, 24]. These works usually vary task difficulty, router policy, or benchmark domain rather than treating skill-library size and geometry as the scaling axis.

Execution-aware agent systems study how intermediate state, feedback, and multi-step interaction change downstream behavior. ReAct [25], Plan-and-Solve [26], Reflexion [27], HuggingGPT [28], MetaGPT [29], and Voyager [30] show that traces, artifacts, feedback, and orchestration can improve end-to-end completion [31, 32, 33, 3, 34]. Our execution law isolates a narrower mechanism: before concrete state appears, multi-skill routing is close to multiplicative composition; after correct execution, realized artifacts can rescue difficult downstream routing.

Our contribution is to connect these strands through empirical laws for skill-library scaling. Prior scaling laws study compute, data, model size, or training allocation [35, 36, 37, 38, 39]; retrieval and representation work explains dense candidate neighborhoods [40, 41, 42, 43]; and the Hick-Hyman law gives a logarithmic analogue for capacity-limited choice [44, 45]. We instead scale the inference-time skill library and show that routing decay, error cascades, black-hole capture, and execution-side rescue are coupled by a unified parameter.

3. Problem Formulation & Experimental Setup

Problem formulation. Let $S_N = \{s_1, \dots, s_N\}$ be the exposed skill library, where each skill has a selectable name, a natural-language description, and an execution interface. Each task q has a gold skill $s^*(q) \in S_N$; that is, the required skill is present in the exposed library by construction. We therefore study availability-preserving failures: the model has access to the necessary skill but may select a wrong in-library alternative. We study two regimes. In a *routing trial*, the model sees only q and the skill descriptions, and the router $\mathcal{R}_\theta$ (LLM θ) proposes

$$\hat{s} = \mathcal{R}_\theta(q, S_N), \quad (1)$$

where $\hat{s}$ may be correct, wrong but in-library, out-of-library, or an abstention. In an *execution trial*, routing and execution are interleaved across K steps or sub-tasks within a pipeline. At step k , the model routes with context c_k and then executes the selected skill: $\hat{s}_k = \mathcal{R}_\theta(q_k, S_N, c_k)$, $y_k = \mathcal{E}_\theta(q_k, \hat{s}_k, c_k)$, where y_k is the realized artifact and c_k contains the task, skill descriptions, and, after execution begins, upstream artifacts $y_{<k}$. Thus routing trials isolate selection before state realization, while execution trials test how realized state changes downstream routing.

The primary routing metric is

$$\text{Acc}(N) = \Pr[\mathcal{R}_\theta(q, S_N) = s^*(q)]. \quad (2)$$

Errors are partitioned into *in-library hijack* ($\hat{s} \in S_N, \hat{s} \neq s^*$), *hallucination* ($\hat{s} \notin S_N$), and abstention. For route-only pipelines, with $s_k^* = s^*(q_k)$,

$$\text{Acc}(N, K) = \Pr[\hat{s}_k = s_k^* \text{ for all } k = 1, \dots, K]. \quad (3)$$

We compare joint success to the multiplicative baseline implied by independent routing. Execution trials additionally score artifact correctness $Q_k \in \{0, 1\}$; in two-step analyses, A denotes the upstream step and B the downstream step, and we measure how correct or wrong state from A changes routing or quality on B . See Appendix A for more mathematical details.

Experimental setup. The scaled library contains 1,141 real-world software-agent skills from public Agent Skills, Claude Code, MCP, and community GitHub repositories after near-duplicate removal [46, 47, 48, 49, 50, 51, 52, 53]. Near-duplicates are removed with BAAI/bge-m3 embeddings [54] (cosine > 0.95), and the resulting skills are organized into 14 software-automation domains. Human annotators constructed 4,075 tasks from predefined skill combinations, with GPT-5.4 assistance only for task synthesis; we retain only tasks with unanimous gold-skill annotations ($\kappa=0.91$, $\sim 9\%$ excluded). In addition, we integrate queries from Terminal-Bench [55], τ -Bench [56], τ^2 -Bench [57], FlowBench [58], PinchBench [52], and related sources, further expanding them and adding tool annotations.

For a size- N routing condition, we include the gold skill for each task and fill the remaining $N-1$ slots with domain-stratified distractors sampled without replacement; this keeps the target distribution fixed while changing library exposure. We use deterministic decoding where provider APIs permit it, parse the first skill-name span returned by the model, and score unmatched names as hallucinations rather than forcing a nearest match. Unless otherwise specified, each case is evaluated an average of four times. Main routing curves sweep $N \in \{10, 20, 50, 100, 200, 500\}$ and $K \in \{1, 2, 3, 5, 10\}$ with $n=500$ domain-stratified tasks per condition. We fit $\text{Acc}(N) = a - b \ln N$ by ordinary least squares over the swept sizes and report Wilson intervals for binomial rates; cross-model correlations use Spearman ρ with percentile bootstrap confidence intervals. Appendix C.1 reports per-model fits, and Appendix C.3 gives retrieval / router baselines and task-difficulty controls.

We evaluate 15 LLMs: GPT-4o-mini [59], GPT-5-mini [60], GPT-5.4-mini [61], GPT-5.4 [62], Claude Sonnet 4.6 [63], Claude Opus 4.6 [64], Gemini 3.1 Pro, Gemini 3.1 Flash Lite [65], GLM-5 [66], GLM-4.7 [67], Kimi K2.5 [68], Kimi K2.6 [69], Doubao Seed 2.0 Pro [70], DeepSeek-V4 Pro [71], and Qwen3-235B [72]. Routing uses plain-text single-skill selection unless stated otherwise, so hallucinations remain measurable. Constrained routing is used only for execution-side isolation and conditions where valid in-library selection must be enforced; it is not pooled

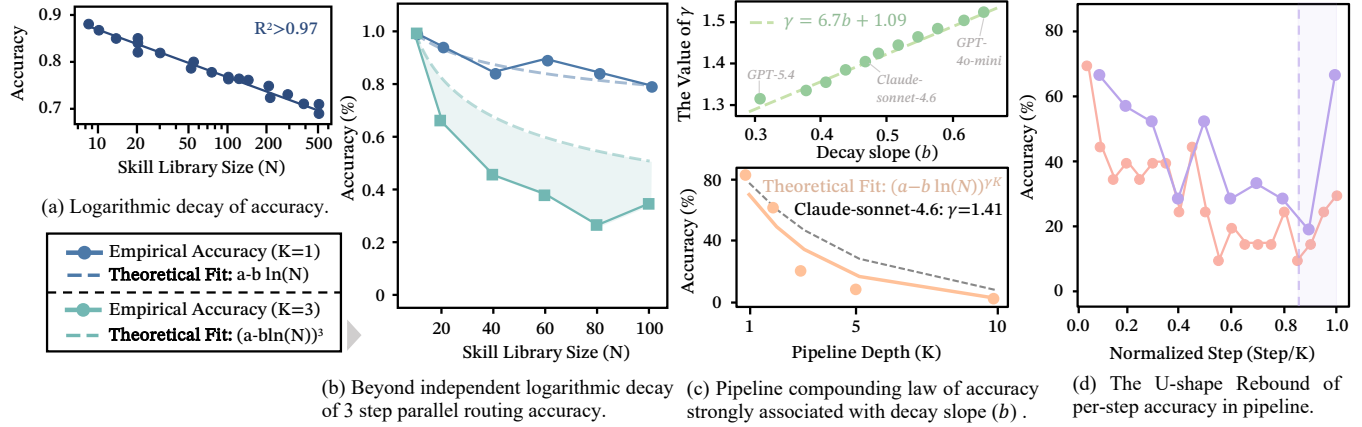


Figure 2: Library scale induces logarithmic single-step decay, super-independent pipeline loss, and a mid-chain routing trough.

into the plain-text hallucination or hijack analyses. Prompts are in Appendix B. Optimization experiments use the same held-out routing protocol as the diagnostic studies.

This section analyzes the routing law before execution: the model observes a task and skill descriptions, chooses $\hat{s}$, and has no execution artifact y_k in context. The central effect is not just that larger libraries contain more candidates; they also create denser neighborhoods of plausible alternatives, so scale changes both the rate and the geometry of routing errors.

We first ask whether increasing N only adds independent choices or changes the structure of the routing problem. For each N , every trial exposes the gold skill plus sampled distractors from the same domain mixture; for route-only pipelines, the K gold skills are fixed and only the exposed distractor set changes. First, single-step routing accuracy decreases almost linearly with $\ln N$ (Fig. 2 (a)). Second, three-step routing plans fall below the accuracy predicted by independently repeating the single-step selector (Fig. 2 (b)). Library scale therefore weakens individual decisions and also makes planned sequences more error-prone.

Together, these results give the first part of the routing law. The logarithmic decay suggests a finite discrimination budget: as N grows, the target skill faces more nearby competitors (Sec. 5). The compounding result explains why this single-step loss becomes worse in planned sequences (Fig. 2 (c)). A wrong early route moves the plan into the wrong semantic neighborhood, making later routes more likely to fail. The U-shape in Fig. 2 (d) then locates the fragile point: middle steps inherit most plausible continuations, while terminal steps partially recover when the final goal narrows the choice set. Append. A & B.7 report mathematical analysis and supplemental cascade diagnostics.

Routing Scaling law. Single-step routing follows $Acc(N) = a - b \ln N$ with $R^2 > 0.97$; each doubling costs ≈ 3 percentage points. In planned route-only pipelines, the loss compounds as $Acc(N, K) \approx (a - b \ln N)^{\gamma K}$ with empirical exponent $\gamma = 6.7b + 1.09$. Per-step accuracy is U-shaped: middle steps are more error-prone than the first or last.

Having established the decay law, we next ask what drives its slope b . Our hypothesis is simple: before execution state is available, routing depends on how clearly the target skill is separated from similar but distinct skills. We test this by varying description quality while holding the task, gold skill, distractor set, and model fixed. More distinguishable descriptions improve accuracy and reduce decay, showing that skill boundaries matter (Fig. 3 (a)).

Boundary quality, however, does not explain all errors. We therefore analyze where the remaining mistakes occur. Fig. 3 (b, c) shows that errors concentrate around intermediate-similarity skills: alternatives that are close enough to be plausible, but not close enough to trigger obvious disambiguation. Exposing more candidates can therefore hurt when it adds these near-miss alternatives.

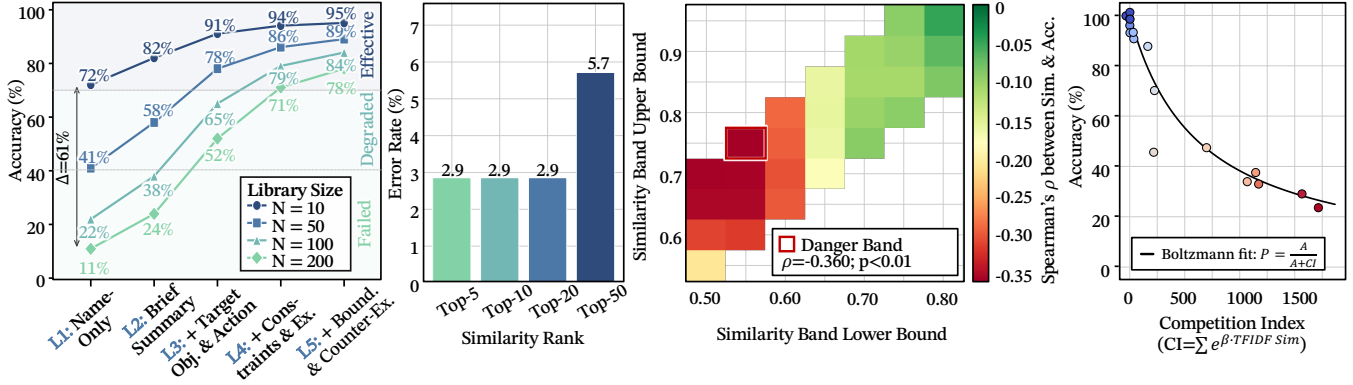


Figure 3: Local Competition Organizes Error. (a) Accuracy decays with N in all description regimes, but better descriptions reduce the decay rate. (b) Top-50 introduces more errors than Top-20/10. (c) Danger band $[0.55, 0.75]$ shows strongest negative ρ with accuracy. (d) The Boltzmann fit between the competition index and accuracy within the danger zone, where $R_{CI}^2 = 0.55$, outperforms the fit with N alone, where $R_N^2 = 0.26$.

Together, these results isolate two sources of routing difficulty: description quality defines the target skill, while local competition reflects nearby alternatives. Hence, global library size is only a coarse proxy: crowded neighborhoods remain difficult despite better descriptions, and CI predicts per-skill failures better than N alone (Fig. 3 (d)). The L4→L5 paradox follows: in small libraries, counterexamples sharpen boundaries; at scale, they may instead foreground plausible confusers and redirect attention to the alternatives they aim to exclude (Append. A & C.2). Appendix C.4 further tests the mechanism by directly manipulating skill-boundary distances.

Semantic competition mechanism. Description quality changes the effective slope: $P(N, \ell) = a_\ell - b_\ell \ln N$, with L1 name-only descriptions at 11–72% and L4 constraint/example descriptions at 71–90% ($b_{L1} = 0.32$ vs. $b_{L4} = 0.08$). Remaining errors are local: 94% stay inside the correct cluster, and the danger zone is intermediate similarity $[0.55, 0.75]$. In this band, $Acc = A/(A + CI)$, and CI predicts errors better than N alone ($R_{CI}^2 = 0.55$ vs. $R_N^2 = 0.26$; see Append. C.4 for the definitions of CI , A , and β).

After identifying what controls the slope, we examine where lost probability mass is redistributed. We construct paired conditions by preserving the underlying task intent while removing concrete nouns, file names, tool-specific verbs, or schema anchors from the query; skill-anchor ablations similarly remove boundary clauses from candidate descriptions while preserving names and interfaces. The pattern is ordered: strong task-query anchors induce local substitution, weak anchors induce in-library drift, and weak anchors combined with vague skills induce “black-hole skill” capture.

In-library drift emerges as task anchors weaken (Fig. 4 (a)). Concrete prompts with strong anchors remain highly accurate at $N = 100$ (89.7%–98.3%). The drift is structured: errors concentrate within the target skill’s functional family and increase with library size as anchors weaken (Fig. 4 (b, c)). This suggests that the model preserves coarse task information but fails to reliably select the correct skill within the same functional cluster. The final stage is black-hole capture, where a few skills become attractors with much higher Gini coefficients (Fig. 4 (d)). As the library grows, broad high-abstraction skills absorb disproportionate routing mass (Fig. 4 (e)). To further characterize this pattern, we analyze routing-mass distributions across skills. Capture occurs only when both skill-anchor removal and vague queries are present (Fig. 4 (f)).

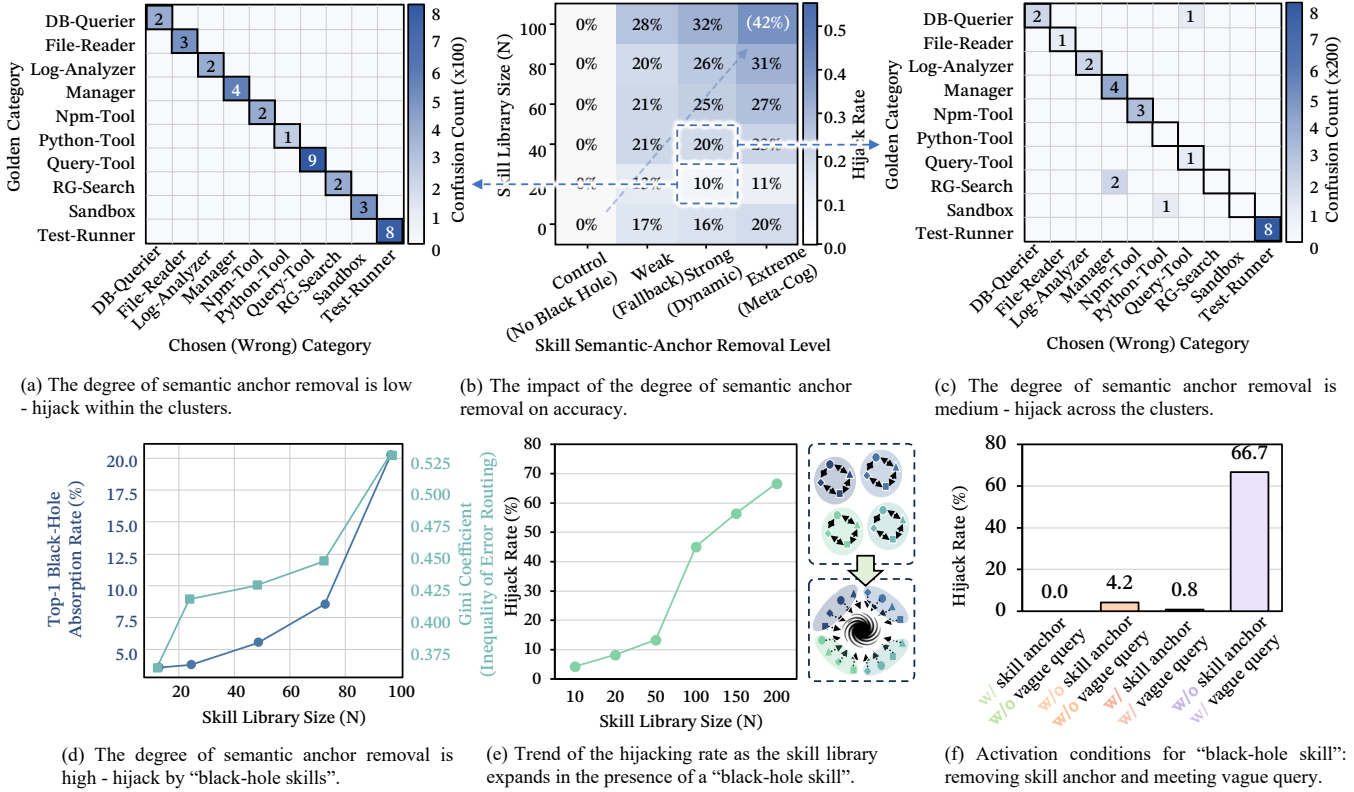


Figure 4: Weak task anchors and vague skill descriptions convert local skill-cluster drift into black-hole skill capture.

Drift and attractor mechanism. With strong anchors, errors are geometrically local substitutions within the same functional family. Weak anchors break this locality: accuracy collapses, but the model still prefers existing skills over hallucinated names. Black-hole capture is conjunctive, not additive: neither weak task anchors nor vague skills are sufficient alone, but together they induce attractor routing.

4. Execution Law: State Realization

The execution law begins where the routing law ends: a selected skill may produce state. We distinguish three cases. First, without state, multi-step skill use reduces to composed routing decisions. Second, after correct execution, the artifact can inform the next decision. Third, if the artifact is incorrect or mismatched, the same state channel may propagate errors or congest context. Thus, the key question is not whether execution helps, but when state alters downstream routing.

4.1. No State Realization: Multiplicative Baseline

We first establish a baseline before any execution artifact is produced. The question is whether routing steps within a pipeline interact merely because they are planned step-by-step. We form ordered two-skill pairs from annotated pipelines, expose the same library to the single-step and paired prompts, and suppress execution artifacts in the paired prompt. If the two routes do not interact, pipeline success should factor as $P(A \cap B) = P(A)P(B)$ (for convenience, we use P to denote the probability of success Acc); any systematic deviation Δ would indicate interaction before state realization. Across 1,690 skill pairs, Δ is near zero (mean < 0.01) across all models (Fig. 5 (a)).

This shows that pipeline prompting alone creates neither reliable positive interaction nor systematic degradation.

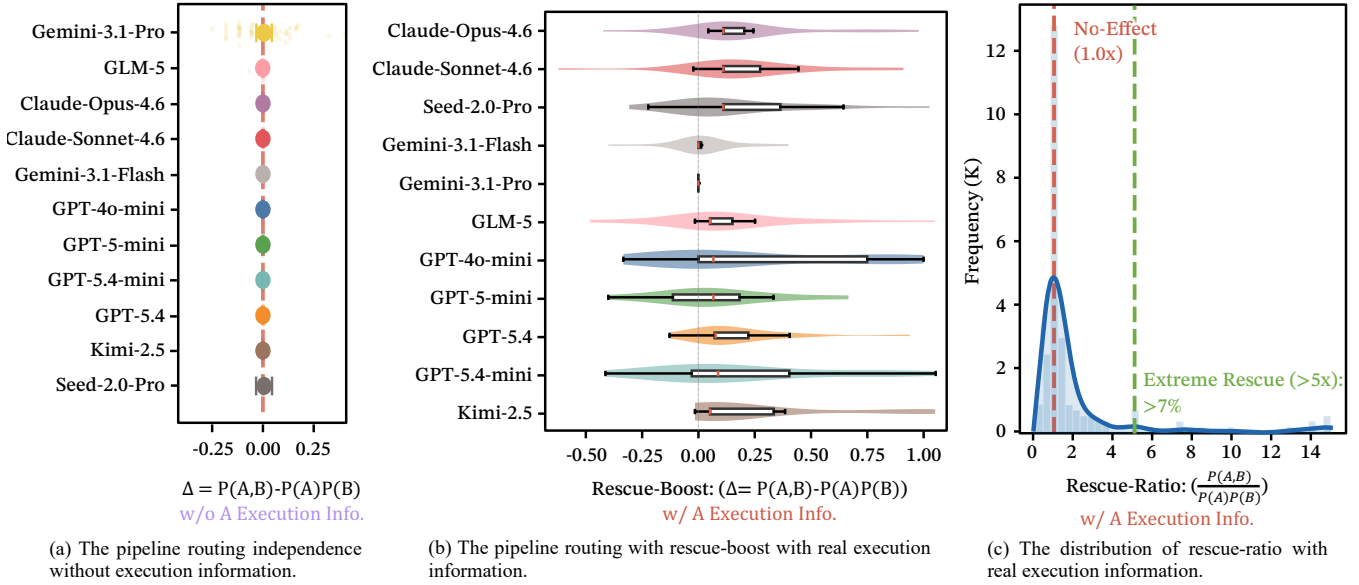


Figure 5: Before state realization, paired routing is approximately multiplicative; after correct execution, realized state rescues difficult downstream routes.

We attribute this to the fact that the router is modeled as a conditional distribution over skills given the prompt, and the no-state isolation setup removes dependence between the two steps before any execution artifact is observed (Appendix A).

No-state multiplicative baseline. Before execution state exists, two-step routing is approximately multiplicative: $P(A \cap B) \approx P(A)P(B)$, $\Delta \equiv P(A \cap B) - P(A)P(B) \approx 0$.

4.2. Correct State Realization: Downstream Rescue

Correct execution changes downstream routing by turning the upstream artifact into concrete evidence for the next skill. In the state-realization condition, we execute the upstream skill, insert its artifact verbatim into the downstream routing context, and keep the downstream gold label fixed relative to the no-state condition (Fig. 5 (b, c)). Rescue should therefore concentrate where upstream execution is correct and pre-state downstream confidence is low.

As shown in Fig. 6 (a), regressing $\Delta P(B|A)$ on $(1 - P(B))P(A)$ yields a slope of $2\alpha \approx 0.76$ across 1,690 pairs. The fit is strongest in the hardest quartile ($R^2=0.81$ vs. 0.54 overall), and the effect size follows the same pattern: hard downstream decisions improve about four times more than median-difficulty ones. This rescue disappears in the no-state condition, showing that the gain is not due to merely pairing two skills. Instead, correct execution turns an underspecified downstream route into a grounded selection problem. We attribute this effect to increased downstream context concreteness, bounded by upstream success and downstream headroom (Appendices A and D.1).

Correct-state rescue law. Correct upstream execution rescues downstream routing in proportion to upstream success and downstream headroom: $\Delta P(B | A) = 2\alpha(1 - P(B))P(A)$, $2\alpha \approx 0.76$.

4.3. Incorrect State Realization: Propagation and Tie-Dependent Synergy

Wrong upstream output has two different effects. We label pair dependency strength κ from the task graph and verify it by whether the downstream checker requires the upstream artifact. If the downstream skill needs that

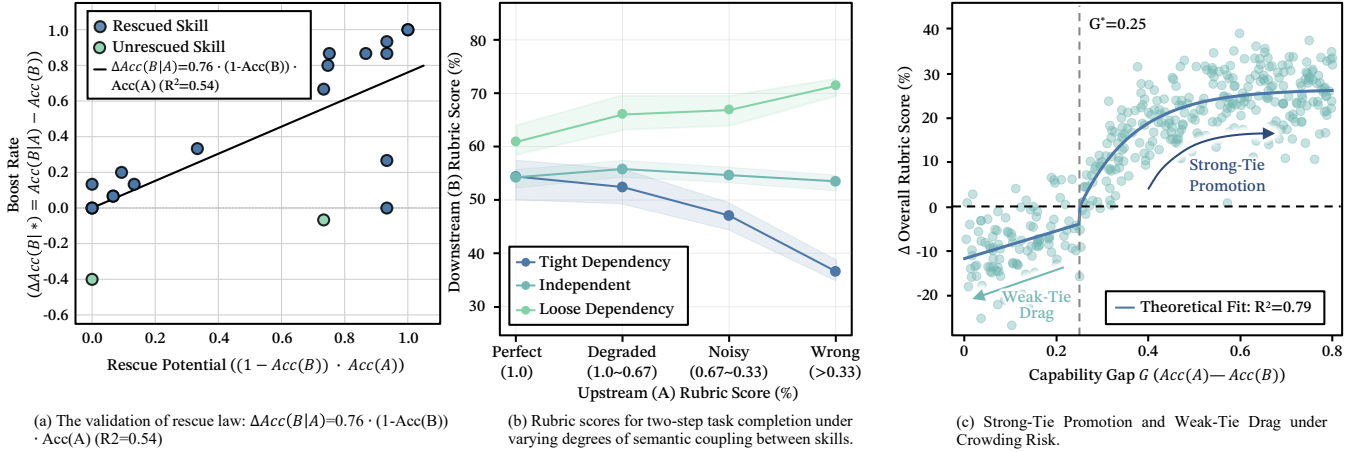


Figure 6: Incorrect state propagates through tight dependencies, while joint execution synergy changes sign with dependency structure and capability gap G .

output, the error is carried forward (Fig. 6 (b)): tight pairs lose over 15% because the required input is missing or incorrect. If the downstream skill does not really need it, the model can ignore the bad output and solve from the downstream task itself: loose pairs gain 2.8%. The no-dependency baseline stays near zero, and ignored-upstream behavior rises by 10–15%, so the gain comes from ignoring the wrong upstream result, not from fixing it.

The same logic explains the capability-gap result. The organizing variable is the capability gap G , not ordering alone. When $G < G^* \approx 0.25$, the pair is a weak tie: neither step provides a strong enough anchor for the other, and joint execution shows a shallow drag that increases toward zero as the gap grows. When $G \geq G^*$, the pair becomes a strong tie: the stronger step provides a usable scaffold for the weaker one, producing a positive gain that rises quickly and then saturates (Fig. 6 (c)). Thus the effect is not a simple strong-first vs. weak-first ordering result; it is a thresholded capability-gap law, with weak-tie drag below G^* and strong-tie promotion above G^* .

Wrong-state propagation and tie-dependent synergy. Wrong upstream state propagates in proportion to dependency strength: tight dependencies carry wrong state forward, while loose dependencies can ignore it and even show a small gain. Joint-execution synergy then changes sign with capability gap G :

$$S(G) \approx \begin{cases} -0.0775 + 0.31 G & G < G^* \approx 0.25 \quad (\text{Weak-Tie}) \\ +0.265 (1 - e^{-(G-G^*)/0.12}) & G \geq G^* \quad (\text{Strong-Tie}) \end{cases} \quad (4)$$

5. A Unified Law Structure for Skill Libraries

The previous two sections establish the routing and execution laws separately. The remaining question is whether the two laws share a state variable. They do: the routing slope b is not only the rate at which a model loses accuracy as the library grows; it also determines how often correct upstream state is available for execution-side rescue. Concretely, take a model whose routing accuracy at exposed library size N is $\text{Acc}(A|N) = a - b \ln N$. This probability also gates whether the downstream step receives useful realized state. Substituting $\text{Acc}(A|N)$ into the independently measured rescue equation from Sec. 4 gives $\text{Acc}_{\text{res}}(B|N) = \text{Acc}(B|N) + 2\alpha (1 - \text{Acc}(B|N)) (a - b \ln N)$, where a, b come from routing experiments, and $2\alpha \approx 0.76$ comes from execution experiments. Yet the prediction tracks independently measured rescue gains across 15

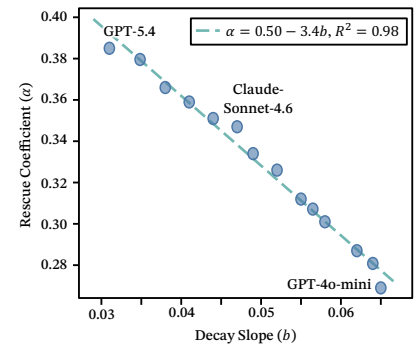


Figure 7: Cross-law coupling: routing slope predicts rescue gain.

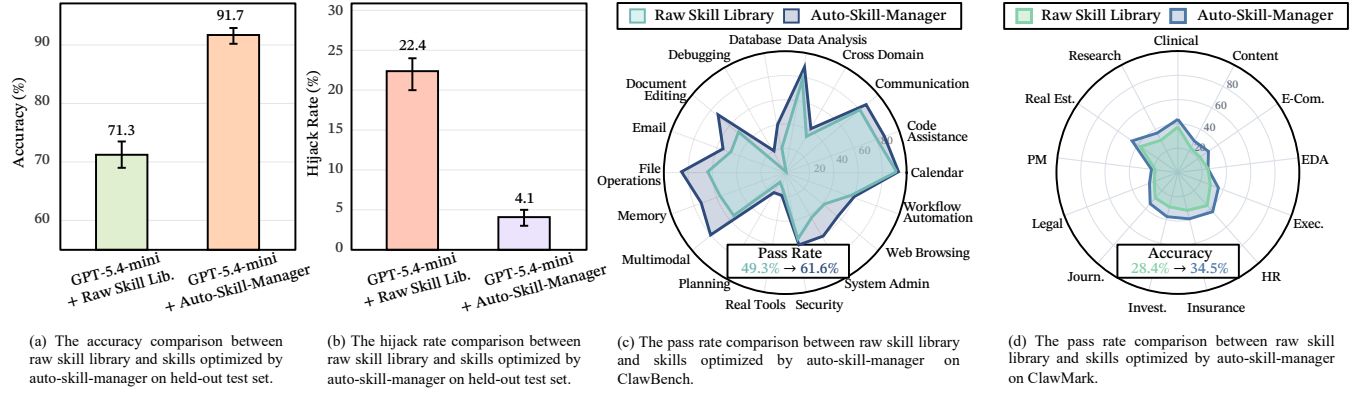


Figure 8: The performance of the law-guided automatic skill manager.

models ($\rho=0.74$, $p<0.001$; Fig. 7). This is the unifying law-level result: b acts as an empirical state variable for skill-library scaling. Reducing it improves immediate routing accuracy and preserves the correct state needed for downstream rescue.

This link also explains why the design optimization in Sec. 6 targets library structure rather than only prompt formatting. Boundary rewriting, nearest-neighbor auditing, and abstract-skill removal all reduce probability mass lost to local competition or attractor capture. In this unified law view, those changes are not separate heuristics: they lower b , delaying the routing cascade and increasing the probability that execution state can help later steps.

6. Law-Guided Auto Skill Manager

The laws make a falsifiable design prediction: if failures are governed by library geometry rather than only by model capability, then editing that geometry should improve a fixed router on held-out tasks. Our law-guided auto skill manager implements this prediction by auditing the exposed library, estimating where routing probability mass is being lost, and editing the library while holding the router fixed. Local competition triggers nearest-neighbor audit and boundary rewrite; cross-cluster drift triggers prompt anchoring; black-hole capture triggers abstract-skill removal or narrowing; and the execution law constrains edits to preserve correct upstream state for downstream rescue. We evaluate prospectively at $N=150$, near the law-predicted transition point for median- b agents, with held-out tasks, gold labels, distractors, model, prompt template, and parser fixed before optimization. Under this matched protocol, the gain is therefore attributable to the library surface and manager edits rather than model retraining or selecting an easier evaluation set.

The prediction holds on both the in-house held-out benchmark and downstream agent benchmarks. On 1,600 held-out tasks, the manager raises accuracy from 71.3% to 91.7% and reduces hijack from 22.4% to 4.1% (Fig. 8 (a, b)). The optimized library transfers to downstream tasks: under the downstream setting, mean pass rate improves from 28.4% to 34.5% on CLAWMARK and from 49.3% to 61.6% on CLAWBENCH (Fig. 8 (c, d); Append. F). Thus the manager is not just a diagnostic cleanup pass, which can also improve routing and positive downstream transfer. Append. E.1–E.5 provide more analysis and discussion.

7. Discussion

Boundaries and improvements. The results should be read as empirical laws for flat natural-language skill libraries, not as universal laws of every possible agent architecture. They also assume availability-preserving evaluation: the required skill is present in the exposed library, so the central error is in-library misselection rather than missing capability. Within this setting, the laws identify concrete improvement levers: report in-library

hijack separately from hallucination, sweep library size N , include weak-anchor prompt conditions, and restructure libraries before deployment.

Broader impacts. The paper turns skill-library scale into a measurable experimental axis, supporting more reproducible studies of routing, execution, and library structure. For deployed agents, the positive impact is safer and more auditable skill exposure; the risk is that the same diagnostics could make harmful automation more reliable, so deployments need access controls, audit logs, and domain-specific safety policies.

Limitations and future work. The exact slopes may change with learned routers, fine-tuning, retrieval architectures, or hierarchical libraries, although the qualitative decay replicates beyond the curated library (Appendix C.5). Future work should repeat the intervention across domains, separate the effects of each design choice, and model when realized state rescues downstream decisions versus when wrong state propagates.

8. Conclusion

This paper identifies empirical scaling laws of skill libraries in settings where the needed skill is available but may be misselected. As reusable skills scale, behavior is shaped not only by model capability but also by library structure. Before execution, routing accuracy follows a logarithmic law; after correct execution, realized state can rescue difficult downstream decisions. A single parameter, the routing decay slope b , links these regimes by predicting both routing collapse and execution-side rescue. We instantiate this coupling in a Law-Guided Auto Skill Manager, showing that the laws can drive practical library optimization with held-out and downstream validation. Large skill libraries should therefore be designed as structured systems, where naming, granularity, exposure, and execution state determine whether scale expands capability or amplifies failure.

References

- [1] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [2] Weikai Xu, Chengrui Huang, Shen Gao, and Shuo Shang. Llm-based agents for tool learning: A survey. *Data Science and Engineering*, 2025.
- [3] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of LLM-based agents. *arXiv preprint arXiv:2503.16416*, 2025.
- [4] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [5] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.
- [6] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
- [7] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023.

- [8] Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, and Yueting Zhuang. TaskBench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*, 2023.
- [9] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [10] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Skill routing for LLM agents at scale. *arXiv preprint arXiv:2603.22455*, 2026.
- [11] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*, 2025.
- [12] Qiguang Chen, Mingda Yang, Libo Qin, Jinhao Liu, Zheng Yan, Jiannan Guan, Dengyun Peng, Yiyan Ji, Hanjing Li, Mengkang Hu, et al. Ai4research: A survey of artificial intelligence for scientific research. *arXiv preprint arXiv:2507.01903*, 2025.
- [13] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.
- [14] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [15] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.
- [16] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junning Zhao, Qian Liu, Che Liu, et al. OpenAgents: An open platform for language agents in the wild. *arXiv preprint arXiv:2310.10634*, 2023.
- [17] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [18] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. SoK: Agentic skills – beyond tool use in LLM agents. *arXiv preprint arXiv:2602.20867*, 2026.
- [19] Lucen Zhong, Zhengxiao Du, Xiaohan Zhang, Haiyi Hu, and Jie Tang. ComplexFuncBench: Exploring multi-step and constrained function calling under long-context scenario. *arXiv preprint arXiv:2501.10132*, 2025.
- [20] Thierry Blankenstein, Jialin Yu, Zixuan Li, Vassilis Plachouras, Sunando Sengupta, Philip Torr, Yarin Gal, Alasdair Paren, and Adel Bibi. BiasBusters: Uncovering and mitigating tool selection bias in large language models. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=DEg4vvE1Yu>.

- [21] Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jieli Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. xRouter: Training cost-aware LLMs orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025.
- [22] Guibin Zhang, Haiyang Yu, Kaiming Yang, Bingli Wu, Fei Huang, Yongbin Li, and Shuicheng Yan. EvoRoute: Experience-driven self-routing LLM agent systems. *arXiv preprint arXiv:2601.02695*, 2026.
- [23] Jingyi Jia and Qinbin Li. Autotool: Efficient tool selection for large language model agents. *arXiv preprint arXiv:2511.14650*, 2025.
- [24] Zheng Fang, Wolfgang Mayer, Zeyu Zhang, Jian Wang, Hong-Yu Zhang, Wanli Li, and Zaiwen Feng. Meta-toolagent: Towards generalizable tool usage in llms through meta-learning. *arXiv preprint arXiv:2601.12680*, 2026.
- [25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [26] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.147. URL <https://aclanthology.org/2023.acl-long.147/>.
- [27] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [28] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving AI tasks with ChatGPT and its friends in Hugging Face. *arXiv preprint arXiv:2303.17580*, 2023.
- [29] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [31] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, et al. TaskWeaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541*, 2023.
- [32] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback. *arXiv preprint arXiv:2309.10691*, 2023.
- [33] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. *arXiv preprint arXiv:2402.04253*, 2024.
- [34] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of llm agents: A survey. *arXiv preprint arXiv:2507.21504*, 2025. URL <https://arxiv.org/abs/2507.21504>.

- [35] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [36] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [37] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- [38] Nicholas Roberts, Niladri S. Chatterji, Sharan Narang, Mike Lewis, and Dieuwke Hupkes. Compute optimal scaling of skills: Knowledge vs reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13295–13316, 2025. doi: 10.18653/v1/2025.findings-acl.688. URL <https://aclanthology.org/2025.findings-acl.688/>.
- [39] Mengsong Wu, Hao Hao, Shuzhen Bi, Keqian Li, Wentao Liu, Siyu Song, Hongbo Zhao, and Aimin Zhou. Scaling laws for educational AI agents. *arXiv preprint arXiv:2603.11709*, 2026. URL <https://arxiv.org/abs/2603.11709>.
- [40] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.
- [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [42] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [43] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023.
- [44] William E. Hick. On the rate of gain of information. *Quarterly Journal of Experimental Psychology*, 4(1):11–26, 1952.
- [45] Ray Hyman. Stimulus information as a determinant of reaction time. *Journal of Experimental Psychology*, 45(3):188–196, 1953.
- [46] Anthropic. Github - anthropics/skills: Public repository for agent skills. <https://github.com/anthropics/skills>, 2025. Accessed: 2026-04-17.
- [47] Anthropic. Claude code. <https://www.anthropic.com/product/claude-code>, 2025. Accessed: 2026-04-16.
- [48] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Accessed: 2026-04-16.
- [49] Trail of Bits. Github - trailofbits/skills: Trail of bits claude code skills for security research, vulnerability detection, and audit workflows. <https://github.com/trailofbits/skills>, 2025. Accessed: 2026-04-16.

- [50] daymade. Claude code skills marketplace. <https://github.com/daymade/claude-code-skills>, 2025. Accessed: 2026-04-16.
- [51] Aaronontheweb. .NET skills for claude code. <https://github.com/Aaronontheweb/dotnet-skills>, 2025. Accessed: 2026-04-16.
- [52] PinchBench Team. PinchBench github organization. <https://github.com/pinchbench>, 2026. Accessed: 2026-04-16.
- [53] Robhunter. Agentdeals. <https://github.com/robhunter/agentdeals>, 2026. Accessed: 2026-04-16.
- [54] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, 2024. doi: 10.18653/v1/2024.findings-acl.137. URL <https://aclanthology.org/2024.findings-acl.137/>.
- [55] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- [56] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [57] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [58] Ruixuan Xiao, Wentao Ma, Ke Wang, Yuchuan Wu, Junbo Zhao, Haobo Wang, Fei Huang, and Yongbin Li. Flow-Bench: Revisiting and benchmarking workflow-guided planning for LLM-based agents. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10883–10900, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.638. URL <https://aclanthology.org/2024.findings-emnlp.638/>.
- [59] OpenAI. Gpt-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024. Accessed: 2026-04-17.
- [60] OpenAI. Gpt-5 system card. OpenAI, August 2025. URL <https://openai.com/index/gpt-5-system-card/>. Accessed: 2026-05-04.
- [61] OpenAI. Introducing gpt-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>, 2026. Accessed: 2026-04-17.
- [62] OpenAI. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>, 2026. Accessed: 2026-04-17.
- [63] Anthropic. Introducing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, 2026.
- [64] Anthropic. Introducing claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026.
- [65] Google DeepMind. Gemini 3. *Google DeepMind*, 2025. URL <https://deepmind.google/models/gemini/>.
- [66] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering, 2026.



- [67] Zhipu AI. Gln-4.7: Advancing the coding capability. <https://z.ai/blog/gln-4.7>, 2025.
- [68] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2.5: Visual agentic intelligence, 2026.
- [69] Moonshot AI. Kimi k2.6: From code to creation, from one to many. <https://www.kimi.com/ai-models/kimi-k2-6>, 2026.
- [70] ByteDance Seed Team. Seed 2.0 official launch. <https://seed.bytedance.com/en/blog/seed2-0-%25E6%25AD%25A3%25E5%25BC%258F%25E5%258F%2591%25E5%25B8%2583>, 2026. Accessed: 2026-04-16.
- [71] DeepSeek-AI. Deepseek-v4:towards highly efficient million-token context intelligence. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, 2026.
- [72] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

Appendix

This appendix is organized as a reader’s map from theory to implementation evidence. It first states the conditional derivations behind the routing and execution laws (Appendix A), then gives the experimental protocol needed to interpret the measurements (Appendix B). The routing-law section reports the fitted decay parameters, the fine-grained law summary, mechanism diagnostics, retriever/router baselines, task-difficulty controls, boundary manipulations, and external validity checks (Appendices C.1–C.5). The execution-law section reports cross-model rescue robustness (Appendix D.1). The final two blocks turn these measurements into system evidence: Appendix E explains the automatic skill manager and its component checks, while Appendix F tests whether the law-guided library changes transfer to downstream skill-agent benchmarks.

A. Mathematical Analysis for the Scaling Laws of Skill Libraries

This section gives the formal backbone behind the empirical laws. The results are conditional theorems: under explicit assumptions about finite-capacity selection, library-competition growth, and state realization, the observed laws follow. The empirical sections then test whether those assumptions are a good description of the measured skill libraries. This separates two claims: the proofs show what the theory implies; the experiments show that the theory’s assumptions hold well enough in the studied regime.

We use N for library size, K for pipeline length, $\text{Acc}(N)$ for routing accuracy, s^* for the target skill, and $\hat{s}$ for the selected skill. The margin $\Delta\mu$ measures how clearly the target skill is separated from its nearest competitors; larger $\Delta\mu$ means easier routing. The competition index CI measures the total pressure from nearby distractor skills. For execution, A is the upstream step, B is the downstream step, and $\Delta\text{Acc}(B | A)$ is the downstream gain after correct upstream execution. The common mechanism is simple: larger candidate sets and smaller margins lower routing confidence, while correct execution state can restore useful context. The shared empirical state variable is the routing decay slope b .

Assumptions. We use the following assumptions throughout the formal statements.

1. **Finite-capacity margin model.** For task q , the router assigns each skill a latent score $Z_i = m_i + \varepsilon_i$, where $m_* - m_j = \Delta_j$ is the semantic margin of the gold skill against distractor j , and ε_i is zero-mean sub-Gaussian noise with scale σ .
2. **Rank-regular clustered library.** Within a functional cluster, distractor skills can be ordered by semantic rank $r = 1, \dots, N-1$ around the target, and their effective overtake weights satisfy $w_r = \kappa/r + O(r^{-1-\xi})$ for some $\kappa, \xi > 0$ over the measured range. This is the scale-free local-crowding condition tested by the CI and semantic-gap diagnostics.
3. **Logarithmic effective competition.** The total effective pressure from plausible distractors is $C(N) = \sum_{j \neq *} w_j(N) = C_0 + C_1 \ln N + o(\ln N)$ over the exposed-library range. Irrelevant distractors have negligible w_j ; local near-misses dominate $C(N)$.
4. **Small-error linearization regime.** Over the measured operating range, the router is not saturated at 0 or 1, so first-order Taylor expansions of the error odds are valid.
5. **State-gated rescue.** Correct upstream execution produces a concrete artifact with rescue coefficient $\alpha \in [0, 1]$; rescue can only occur when upstream routing/execution is correct and only to the extent that the downstream route has remaining headroom.
6. **No pre-state leakage.** In the no-state condition, paired routing prompts do not contain an execution artifact, so any interaction before execution is bounded by a protocol term ϵ_{proto} .

Lemma 1 (Clustered libraries imply logarithmic effective competition). Under Assumption 2, Assumption 3 follows with $C_1 = \kappa$.

Table 1: Fine-grained appendix laws underlying the grouped routing-law and execution-law summaries in the main text.

Regime	Law	Variable	Observed Law & Pattern
Routing	Law 1: Logarithmic Decay	Capacity-limited selection	$a - b \ln N$
Routing	Law 2: Pipeline Compounding	Local errors corrupt downstream context	$(a - b \ln N)^{y_K}$
Routing	Law 3: Pipeline Rebound	Information compresses at mid-chain	$\delta(N) = \delta_0 + c_\delta \ln N$
Routing	Law 4: Description Quality	Quality changes effective semantic margin	$a_\ell - b_\ell \ln N$
Routing	Law 5: Local Competition	Confusion peaks at intermediate similarity	Peak at [0.55, 0.75]
Routing	Law 6: Failure Geometry	Description scope induces directional confusion	Local family substitution
Routing	Law 7: Anchor Removal	Vague prompts reduce semantic separation	Cross-cluster drift
Routing	Law 8: Dual-Trigger Black Hole	Abstract skills become competitive near weak anchors	Conjunctive attractor capture
Execution	Law 9: Routing Independence	No state $\Rightarrow$ multiplicative routing	$Acc(A, B) \approx Acc(A)Acc(B)$
Execution	Law 10: Execution Rescue	Realized artifacts increase concreteness	$\Delta Acc(B A) \propto (1 - Acc(B))Acc(A)$
Execution	Law 11: Context-Conditioned Recovery	Coupling type determines harm vs. help	$-0.072\kappa + 0.028(1 - \kappa)$
Execution	Law 12: Strong-Tie Promotion / Weak-Tie Drag	Capability gap governs sign and saturation of joint synergy	Piecewise $S(G)$ with $G^* \approx 0.25$

Proof. By Assumption 2,

$$C(N) = \sum_{r=1}^{N-1} w_r = \kappa \sum_{r=1}^{N-1} \frac{1}{r} + \sum_{r=1}^{N-1} O(r^{-1-\xi}). \quad (5)$$

The harmonic sum satisfies $\sum_{r=1}^{N-1} 1/r = \ln N + \gamma_E + o(1)$, where γ_E is the Euler constant. The residual series $\sum_r O(r^{-1-\xi})$ converges because $\xi > 0$, so it contributes a constant plus $o(1)$. Therefore

$$C(N) = C_0 + \kappa \ln N + o(\ln N), \quad (6)$$

which is Assumption 3. $\square$

Proposition 1 (Routing law from finite-capacity competition). Under Assumptions 1–4, there exist constants a and $b > 0$ such that

$$Acc(N) = a - b \ln N + o(\ln N) \quad (7)$$

over the measured library range. This is a finite-range local approximation, not a universal asymptotic claim. Moreover, increasing semantic margins decreases b .

Proof. The gold skill is selected when $Z_\star > Z_j$ for all distractors. For each distractor, define its pairwise overtake probability

$$r_j = \Pr[Z_j \geq Z_\star] = \Pr[\varepsilon_j - \varepsilon_\star \geq \Delta_j]. \quad (8)$$

Sub-Gaussian concentration gives $r_j \leq \exp(-\Delta_j^2/(4\sigma^2))$ up to a constant factor. In the measured operating range, we assume the effective competition weights are comparable to these pairwise overtake probabilities, so that the same local crowding index controls both the upper bound and the observed error pressure. Write this effective pressure as w_j . By the union bound and the usual inclusion–exclusion expansion,

$$\Pr[\exists j : Z_j \geq Z_*] = \sum_{j \neq *} r_j + O\left(\left(\sum_{j \neq *} r_j\right)^2\right). \quad (9)$$

Assumption 3 and the comparability condition give $\sum_j r_j = \lambda_0 + \lambda_1 \ln N + o(\ln N)$ after absorbing constant factors into λ_0, λ_1 . Assumption 4 makes the second-order term bounded and locally linearizable in the measured range, so

$$\text{Acc}(N) = 1 - \Pr[\exists j : Z_j \geq Z_*] = a - b \ln N + o(\ln N). \quad (10)$$

If descriptions or skill boundaries increase margins Δ_j , the effective overtake pressures decrease under the same comparability condition, so the log-growth coefficient λ_1 and hence b decrease. This proves the local logarithmic form and the design prediction that sharpening boundaries reduces the slope in the measured regime. $\square$

Lemma 2 (Boundary interventions identify semantic competition). Consider two candidate mechanisms for the decay slope:

$$b = b_{\text{ctx}}(L_N) + b_{\text{sem}}(\{\Delta_j\}), \quad (11)$$

where b_{ctx} depends only on prompt length or number of exposed tokens L_N , and b_{sem} depends on semantic margins. A boundary rewrite that preserves token length and candidate set but increases margins Δ_j can reduce b only through b_{sem} . Therefore, a statistically reliable decrease in b under length-matched boundary rewrites rules out a pure context-length explanation $b = b_{\text{ctx}}(L_N)$.

Proof. In the intervention, L_N and the exposed candidate set are fixed by construction, so $b_{\text{ctx}}(L_N)$ is invariant. The only changed argument in the decomposition is the margin set $\{\Delta_j\}$. By Proposition A, increasing margins decreases the effective overtake pressures under the finite-range comparability condition, and hence decreases b_{sem} . Thus any observed slope change under a length-matched boundary rewrite is identified as semantic-competition movement within this additive model. If b were purely a function of context length, the treatment effect would be exactly zero. $\square$

Proposition 2 (Pipeline compounding from context compression). Let $p_N = a - b \ln N$ be the isolated single-step routing probability. In a route-only pipeline, suppose each downstream step after the first pays a compression penalty $0 \leq \eta_N < p_N$ because the prompt contains an intermediate plan state rather than the full original task anchor. Then strict all-correct pipeline success is

$$\text{Acc}(N, K) = p_N(p_N - \eta_N)^{K-1}. \quad (12)$$

If $\eta_N > 0$, then $\text{Acc}(N, K) < p_N^K$, so pipeline success falls below independent repetition. If $\eta_N/p_N = O(b)$, then

$$\text{Acc}(N, K) = p_N^K \exp\left[-(K-1)\eta_N/p_N + O(K(\eta_N/p_N)^2)\right], \quad (13)$$

which can be reparameterized as the empirical exponent form $\text{Acc}(N, K) \approx p_N^{\gamma K}$ with $\gamma > 1$. The fitted γ increases with routing fragility when the relative compression penalty η_N/p_N increases sufficiently with b .

Proof. Let $\delta_k = 1$ denote a correct route. The first step sees the original task anchor, so $\Pr(\delta_1 = 1) = p_N$. Each later step is routed from a compressed intermediate context, so

$$\Pr(\delta_k = 1 \mid \delta_1 = \dots = \delta_{k-1} = 1) = p_N - \eta_N, \quad k \geq 2. \quad (14)$$

Multiplying conditional probabilities gives $p_N(p_N - \eta_N)^{K-1}$. Since $p_N - \eta_N < p_N$ when $\eta_N > 0$, the product is strictly below p_N^K . For the exponential form,

$$p_N(p_N - \eta_N)^{K-1} = p_N^K(1 - \eta_N/p_N)^{K-1}. \quad (15)$$

Using $\ln(1 - x) = -x + O(x^2)$ gives the displayed expansion. Finally, because $p_N^{yK} = p_N^K \exp((y - 1)K \ln p_N)$ and $\ln p_N < 0$, the additional exponential loss corresponds to some $\gamma > 1$. Cross-model monotonicity of γ in b is therefore an empirical regularity unless one additionally assumes that the growth of η_N/p_N dominates the change in $\ln p_N$. $\square$

Proposition 3 (Execution rescue). Under Assumption 5, the downstream improvement from correct upstream state has the form

$$\Delta \text{Acc}(B \mid A) = 2\alpha \text{Acc}(A)(1 - \text{Acc}(B)). \quad (16)$$

Proof. Rescue requires two events: correct upstream state exists, and downstream routing has remaining headroom. The first event has probability $\text{Acc}(A)$. Conditional on useful state, the largest possible downstream improvement is bounded by $1 - \text{Acc}(B)$, because routes already correct before state cannot improve. Let α be the fraction of this available headroom converted into correct downstream decisions by the artifact. Multiplying the gate $\text{Acc}(A)$, the available headroom $1 - \text{Acc}(B)$, and the conversion coefficient α yields the claim. $\square$

Theorem 1 (Cross-law coupling through the routing slope). Under Assumptions 1–5 and Propositions A–A, the execution-side rescued downstream accuracy satisfies

$$\text{Acc}_{\text{res}}(B) = \text{Acc}(B) + 2\alpha(1 - \text{Acc}(B))(a - b \ln N) + o(\ln N). \quad (17)$$

Thus, holding a, N, α , and $\text{Acc}(B)$ fixed, larger routing decay slope b strictly decreases the expected execution rescue.

Coupling diagnostic details. The coupling check deliberately uses two measurements that are estimated from different experimental regimes. For each model, the routing-side quantity is the slope b from the single-step library-size sweep in Appendix C.1. The execution-side quantity is the rescue coefficient α from the correct-state execution probes, estimated by regressing downstream rescue on upstream correctness and downstream headroom. The same ordered skill pairs used for rescue estimation are not used to refit the routing slope.

After these two quantities are fixed for all 15 models, we run a cross-model diagnostic regression of α on b and obtain $\alpha = 0.50 - 3.4b$. The purpose is not to add a new free parameter to the main law, but to test whether the independently measured routing fragility of a model predicts how much correct execution state can rescue later decisions. We therefore interpret the relation as a no-refit consistency check linking the routing and execution experiments. The derivative in the theorem is a partial derivative with α held fixed; if one instead treats the empirical diagnostic $\alpha = \alpha(b)$ as a structural relation, the total derivative contains the additional term $\alpha'(b)(a - b \ln N)$.

Proof. By Proposition A, the probability that the upstream route supplies the correct state is $Acc(A) = a - b \ln N + o(\ln N)$. Substitute this into Proposition A:

$$\Delta Acc(B | A) = 2\alpha(1 - Acc(B))(a - b \ln N) + o(\ln N). \quad (18)$$

Adding this improvement to the pre-state downstream accuracy $Acc(B)$ gives the stated equation. Differentiating with respect to b gives

$$\frac{\partial Acc_{res}(B)}{\partial b} = -2\alpha(1 - Acc(B)) \ln N < 0 \quad (19)$$

for $\alpha > 0$, $Acc(B) < 1$, and $N > 1$. Hence the same slope that governs routing collapse also controls the expected gate through which correct upstream state can rescue downstream routing. $\square$

L1: logarithmic decay from capacity-limited selection. L1 is the direct consequence of Proposition A. A complementary finite-range view comes from an extreme-value approximation. Let the target skill score be $z^* = \langle \mathbf{u}, \tilde{\mathbf{v}}^* \rangle$ and each competing score be $z_j = \langle \mathbf{u}, \tilde{\mathbf{v}}_j \rangle$, where $\tilde{\mathbf{v}}_i = \mathbf{v}_i + \epsilon_i$ is the noisy representation available to a finite-capacity router. If the nearest semantic margin is $\Delta\mu$ and the effective noise variance is $\sigma_0^2 f(N)$, then

$$Acc(N) = \Pr[z^* > \max_{j \neq *} z_j] \approx \Phi\left(\frac{\Delta\mu - \sigma_0 \sqrt{f(N)} \sqrt{2 \ln N}}{\sigma_0 \sqrt{f(N)}}\right). \quad (20)$$

This expression is not used as a global asymptotic equality. In the finite range $N \in \{10, \dots, 500\}$, define $x = \ln N$ and write the right-hand side as $F(x)$. Taylor expansion around the center of the sweep x_0 gives

$$F(x) = F(x_0) + F'(x_0)(x - x_0) + O((x - x_0)^2) = a - b \ln N + O((\ln N - \ln N_0)^2), \quad (21)$$

where $b = -F'(x_0) > 0$ in the measured decreasing regime. The derivative $F'(x_0)$ still depends on the local value of $f(N)$ and on the expansion point; we use this approximation only to justify a local slope. The monotone margin prediction comes from Proposition A: larger semantic separation lowers effective overtake pressure and therefore lowers the fitted log-slope in the measured regime. Thus the proof claim is local: finite-capacity extreme competition plus observed log-scale crowding yields the measured $a - b \ln N$ form.

Assumption notes. The scaling $f(N) \propto \ln N$ is an empirical approximation, motivated by the sub-linear growth of $CI(N)$ in our 14-cluster library and supported by the AIC comparison ($\Delta AIC > 8$ across all 15 models). The Gaussian approximation treats noise terms ϵ_i as independent; this is imperfect for semantically similar skills, but the resulting prediction is supported by the per-skill CI regression ($R^2 = 0.55$, Sec. 3) and by the cross-model log fits ($R^2 > 0.97$ for all models). If added skills were mostly irrelevant, CI would not grow with N and the log slope would collapse toward zero; if the effect were pure context length, description rewrites would not selectively reduce b for crowded neighborhoods. Both alternatives are contradicted by the semantic-gap regression and the boundary manipulation in Appendix C.4.

L2: pipeline compounding and cascade amplification. L2 follows from Proposition A. Let $\delta_k \in \{0, 1\}$ denote whether step k is routed correctly, and let $p_N = Acc(N)$. The independence baseline is

$$P_{ind}(N, K) = \prod_{k=1}^K \Pr(\delta_k = 1) = p_N^K. \quad (22)$$

In a route-only plan, later prompts are conditioned on compressed planner state. Model that loss by

$$\Pr(\delta_k = 1 \mid \delta_1 = \dots = \delta_{k-1} = 1) = p_N - \eta_N, \quad k \geq 2, \quad (23)$$

with $0 \leq \eta_N < p_N$. Multiplying the chain-rule factors gives

$$Acc(N, K) = p_N(p_N - \eta_N)^{K-1}, \quad p_N = Acc(N), \quad 0 \leq \eta_N < p_N, \quad (24)$$

and hence

$$\frac{Acc(N, K)}{P_{ind}(N, K)} = \left(1 - \frac{\eta_N}{p_N}\right)^{K-1} < 1 \quad (25)$$

whenever $\eta_N > 0$. Taking logs,

$$\ln Acc(N, K) = K \ln p_N + (K - 1) \ln(1 - \eta_N/p_N) = K \ln p_N - (K - 1)\eta_N/p_N + O(K\eta_N^2/p_N^2). \quad (26)$$

Equivalently, define

$$\gamma = 1 + \frac{K - 1}{K} \frac{\ln(1 - \eta_N/p_N)}{\ln p_N}, \quad (27)$$

which is larger than one because both logarithms are negative. Then $Acc(N, K) = p_N^{\gamma K}$ exactly for fixed N, K, η_N . The empirical scaling law uses this as a low-dimensional summary across the measured N range. The derivation is what establishes compounding below the independence baseline.

L3: mid-chain fragility from information compression. L3 follows by adding terminal recovery to the same compression model. Let I_k be the mutual information between the original user intent and the context at step k . Use

$$I_k = I_0 - c_0(k - 1) \ln N + r_k, \quad (28)$$

where $c_0 \ln N$ is per-step identification loss and r_k is a terminal-constraint recovery term. Assume r_k is negligible before the final segment and increases near the end. For the middle step $m = \lfloor K/2 \rfloor$,

$$I_1 - I_m = c_0(m - 1) \ln N - (r_m - r_1) \approx c_0(m - 1) \ln N. \quad (29)$$

For a late step K , the terminal constraint contributes $r_K > r_m$, so

$$I_K - I_m = c_0(m - K) \ln N + (r_K - r_m). \quad (30)$$

A rebound occurs when $r_K - r_m > c_0(K - m) \ln N$; its depth is the recovered accuracy gap between the middle and late steps. In the finite range, write the relative terminal recovery as $r_K - r_m = r_0 + r_1 \ln N$, where r_1 captures the fact that final-step constraints become more valuable when the intermediate library ambiguity is larger. If local routing accuracy satisfies $Acc_k \approx \beta_0 + \beta_1 I_k$, then the rebound depth is

$$\delta(N) = Acc_K - Acc_m \approx \beta_1 r_0 + \beta_1 \{r_1 - c_0(K - m)\} \ln N. \quad (31)$$

Thus $\delta(N) = \delta_0 + c_\delta \ln N$ with $c_\delta > 0$ whenever $r_1 > c_0(K - m)$. The proof is conditional on the explicit recovery term; monotonicity alone would not imply the rebound.

L4: description quality as effective semantic margin. L4 follows from the margin dependence in Proposition A. Model the description-quality effect as a saturating margin gain:

$$\Delta\mu(\ell) = \Delta\mu_0 \frac{\ell}{\ell + c}, \quad (32)$$

where ℓ is description quality and the denominator says that each extra detail helps less after the main boundary is already clear. For a distractor with margin $\Delta_j(\ell)$, the sub-Gaussian bound gives

$$r_j(\ell) \leq \exp\{-\Delta_j(\ell)^2/(4\sigma^2)\}. \quad (33)$$

An upper bound alone does not identify the derivative of r_j . We therefore use the same finite-range comparability condition as Proposition A: in the measured regime, the effective overtake pressure is

$$r_j(\ell) = \theta_j(\ell) \exp\{-\Delta_j(\ell)^2/(4\sigma^2)\}, \quad (34)$$

where $\theta_j(\ell)$ is bounded and does not increase fast enough to offset margin gain. Then

$$\frac{\partial \log r_j}{\partial \ell} = \frac{\partial \log \theta_j}{\partial \ell} - \frac{\Delta_j(\ell)\Delta_j'(\ell)}{2\sigma^2} < 0 \quad (35)$$

whenever the margin term dominates. Since the routing slope $b(\ell)$ is the local log-growth coefficient of $\sum_j r_j(\ell)$ in Proposition A, this gives $b'(\ell) < 0$ in that regime. The induced routing curve is

$$\text{Acc}(N, \ell) \approx a(\ell) - b(\ell) \ln N, \quad b'(\ell) < 0, \quad (36)$$

with $a(\ell)$ allowed to move slightly because clearer descriptions can also improve small-library routing. Thus the improvement is derived through lower overtake pressure, not by multiplying the whole accuracy curve by a description-quality factor.

L5: danger zone and non-monotonic competition. L5 can be derived by separating semantic similarity from explicit disambiguation. Let $d = \Delta\mu_{ij}$ be the distance between two candidate descriptions. Write confusion pressure as

$$\Pr(i \rightarrow j) \propto g(d) \exp(-d^2/2\sigma^2), \quad (37)$$

where the Gaussian term is semantic substitutability and $g(d)$ is the probability that the candidate survives relevance and duplicate checks. Assume $g(0) = 0$ because exact or near duplicates trigger explicit boundary checking, $g'(d) > 0$ for small d , and $g(d)$ is positive and differentiable on $(0, \infty)$, bounded, and not exponentially increasing. We also assume the duplicate-check release is steep enough near zero that $g'(d)/g(d) > d/\sigma^2$ for sufficiently small positive d . The derivative of log confusion pressure is

$$\frac{d}{dd} \log \Pr(i \rightarrow j) = \frac{g'(d)}{g(d)} - \frac{d}{\sigma^2}. \quad (38)$$

For small $d > 0$, the release condition makes confusion rise. For large d , the boundedness condition makes the Gaussian tail dominate, so confusion tends to zero. By continuity, the maximum is attained at an interior point d^* ; any differentiable interior maximizer satisfies the first-order condition $g'(d^*)/g(d^*) = d^*/\sigma^2$. This gives the non-monotonic danger zone under the duplicate-check model: plausible near-misses are worse than both irrelevant skills and obvious duplicates.

L6: descriptor geometry and asymmetric confusion. L6 follows from a set-overlap model of descriptor scope. Treat description d_i as defining a task region C_i . The overlap term is

$$J(C_i, C_j) = \frac{|C_i \cap C_j|}{|C_i \cup C_j|}, \quad (39)$$

and the directional asymmetry term is

$$A(d_i, d_j) = \frac{|C_i \setminus C_j| - |C_j \setminus C_i|}{|C_i \cup C_j|} \quad (40)$$

with $|A| \leq 1$. This is a signed coverage-imbalance term, not an overlap measure. Define mutual-confusion risk as

$$\text{Risk}(i, j) \propto J(C_i, C_j)(1 - |A(d_i, d_j)|). \quad (41)$$

Then $\partial \text{Risk} / \partial J > 0$ when $|A| < 1$, and $\partial \text{Risk} / \partial |A| = -J < 0$. Thus high overlap increases confusion, while directional ownership decreases it. Boundary rewrites reduce risk either by lowering $J(C_i, C_j)$ or by increasing $|A|$ through clearer scope boundaries.

L7–L8: anchor removal and black-hole capture. L7–L8 explain why vague prompts and broad skills are dangerous mainly together. A concrete prompt points toward a specific skill, so the correct skill keeps a large margin. A vague prompt points toward the center of a functional cluster instead. If the library contains a broad abstract skill near that center, the broad skill can become the easiest explanation of the prompt:

$$\text{score}(s_{\text{abs}}) \approx \frac{1}{M} \sum_{i=1}^M \langle \mathbf{u}_{\text{vague}}, \mathbf{v}_i \rangle + \epsilon, \quad (42)$$

where M is the number of concrete skills whose intent the abstract skill subsumes. Vague prompts alone need not cause capture, and abstract skills alone need not cause capture. The failure appears when both push probability mass toward the same broad skill.

Anchor-loss and black-hole experiment details. The anchor-loss experiment starts from matched task intents and progressively removes information that normally identifies the target skill. Query-anchor removal deletes or generalizes concrete nouns, file names, API names, schema fields, and domain-specific verbs while preserving the intended operation. Skill-anchor removal weakens the candidate descriptions by removing boundary clauses or replacing specific operational constraints with broader functional language. Because the task intent and gold skill are retained, a drop in accuracy measures loss of routing evidence rather than a change in what the task requires.

For every in-library error, we record both the gold functional category and the chosen wrong category. This lets us distinguish three regimes: local substitution within the same cluster, cross-cluster drift, and capture by a broad abstract skill. The semantic-anchor removal sweep crosses library size with removal level and measures hijack rate, keeping hallucinated skill names separate from in-library errors. The black-hole analysis then retains a broad catch-all skill in the exposed pool and measures top-1 absorption, routing-mass Gini, and hijack rate as the library grows. Finally, the activation test crosses two factors, skill-anchor removal and vague query construction. We treat capture as a dual-trigger effect only if the combined condition exceeds both single-factor controls; this is the empirical counterpart of Proposition 4.

Proposition 4 (Dual-trigger capture). Let the gold-skill score margin over an abstract skill be

$$M = M_0 - \Delta_q - \Delta_s, \quad (43)$$

where $M_0 > 0$ is the concrete-query/concrete-skill margin, $\Delta_q \geq 0$ is the margin loss from query-anchor removal, and $\Delta_s \geq 0$ is the margin loss from broad abstract skill descriptions. If capture occurs when $M < 0$, then there exist regimes where neither trigger alone causes capture but both together do:

$$M_0 - \Delta_q > 0, \quad M_0 - \Delta_s > 0, \quad M_0 - \Delta_q - \Delta_s < 0. \quad (44)$$

Proof. The three inequalities are jointly feasible whenever $\max(\Delta_q, \Delta_s) < M_0 < \Delta_q + \Delta_s$. In that interval, each individual manipulation leaves the gold skill ahead of the abstract skill, but their combined margin loss reverses the ordering. Thus capture is an interaction term in margin space rather than a necessary consequence of either vague prompts or abstract skills alone. $\square$

L9: pre-state routing independence. L9 is the clean baseline for execution: before anything runs, there is no artifact from step A that can help step B . Let A and B be the events that the upstream and downstream routes are correct. Once we condition on the two task prompts and the exposed library, the downstream route has no extra state from the upstream route. The no-state baseline is therefore

$$\Pr(A \cap B \mid q_A, q_B, S_N) = \Pr(A \mid q_A, S_N) \Pr(B \mid q_B, S_N) + \epsilon_{\text{proto}}. \quad (45)$$

Here ϵ_{proto} captures residual prompt-format correlations, shared task difficulty, or cluster-level biases that are not real execution effects. The no-state isolation setup estimates this term and finds it small in the measured regime, so multiplicativity is an empirical baseline rather than a theorem from the mere absence of an artifact.

L10: execution rescue from concrete state. L10 says that execution helps only after it produces useful concrete state. Let o_A be the artifact from the upstream step. If o_A tells the router something about the correct downstream skill s_B^* , then the downstream prompt becomes less ambiguous:

$$C(q_B, o_A) = C(q_B) + I(s_B^*; o_A | q_B), \quad (46)$$

where $C(\cdot)$ is concreteness and the mutual-information term is the new evidence supplied by the artifact. Closing this evidence model with the state-gated headroom model in Proposition A gives $\Delta \text{Acc}(B | A) = 2\alpha \text{Acc}(A)(1 - \text{Acc}(B))$. $\text{Acc}(A)$ appears because rescue requires correct upstream state; $(1 - \text{Acc}(B))$ appears because easy downstream routes have little headroom.

L11: dependency-weighted propagation. L11 describes what happens when the upstream state is wrong. The key quantity is dependency strength: if downstream step truly needs the upstream artifact, wrong state should hurt; if dependency is loose, LLM may ignore the bad artifact and use the original task signal. Let $\kappa \in [0, 1]$ measure how much downstream evidence must come from the upstream artifact. Then the quality change decomposes into propagated loss and task-context recovery:

$$\Delta Q_B(\kappa) = -\lambda \kappa + r(1 - \kappa). \quad (47)$$

Estimating λ and r from tight and loose dependency groups yields the reported propagation relation. The no-dependency baseline constrains r : recovery requires the downstream task to remain understandable without the upstream artifact, not arbitrary unrelated state. Wrong state hurts when downstream step needs it, but can be ignored when downstream task remains identifiable without it.

Proposition 5 (Dependency-weighted wrong-state propagation). Suppose a wrong upstream artifact is used by the downstream step with dependency weight $\kappa \in [0, 1]$. Let $\lambda > 0$ be the loss when required wrong state is used, and let $r \geq 0$ be the recovery gain from ignoring irrelevant wrong state and relying on the downstream task context. Then

$$\Delta Q_B(\kappa) = -\lambda \kappa + r(1 - \kappa). \quad (48)$$

The effect is negative for $\kappa > r/(\lambda + r)$ and positive for $\kappa < r/(\lambda + r)$.

Proof. Decompose the downstream decision into two mutually exclusive channels: with weight κ , the checker-relevant evidence must come from the upstream artifact; with weight $1 - \kappa$, the downstream task remains identifiable without that artifact. Wrong required state contributes expected change $-\lambda$ on the first channel. Ignoring irrelevant wrong state contributes expected recovery r on the second channel. Linearity of expectation gives the stated affine form. Solving $-\lambda \kappa + r(1 - \kappa) = 0$ gives the sign threshold $\kappa = r/(\lambda + r)$. $\square$

L12: strong-tie promotion and weak-tie drag. L12 explains why putting two steps together can either help or hurt depending on the capability gap $G = |\text{Acc}(A) - \text{Acc}(B)|$. When $G < G^*$, the pair is effectively weak-tie: neither step gives the other a strong enough anchor, so joint execution carries a small drag that shrinks as the gap grows. When $G \geq G^*$, the pair becomes strong-tie: the stronger step can promote the weaker one, and the gain saturates once the useful scaffold has been provided. The empirical fit is

$$S(G) \approx \begin{cases} -0.0775 + 0.31 G & G < G^*, \\ +0.265 (1 - e^{-(G-G^*)/0.12}) & G \geq G^*, \end{cases} \quad G^* \approx 0.25. \quad (49)$$

Thus Law 12 is not a symmetric ordering effect. The negative weak-tie regime is shallow, while the positive strong-tie regime rises quickly and saturates, matching the main-text observation that promotion by a sufficiently stronger step dominates weak-tie drag.

Proposition 6 (Strong-tie promotion threshold). Let joint execution have crowding cost $c(G)$ that decreases approximately linearly with capability gap in the weak-tie regime, and scaffold benefit $h(G)$ that is zero below a threshold G^* and saturates above it:

$$c(G) = (c_0 - c_1 G)_+, \quad h(G) = h_{\max}(1 - e^{-(G-G^*)/\tau})\mathbf{1}\{G \geq G^*\}. \quad (50)$$

Then the net synergy $S(G) = h(G) - c(G)$ is negative-but-increasing below G^* and becomes positive-saturating once the scaffold benefit exceeds the residual cost, matching the empirical form in Law 12 in main text.

Proof. For $G < G^*$, $h(G) = 0$, so $S(G) = -c_0 + c_1 G$ as long as the positive part is active, a shallow negative line whenever $0 < c_1 G < c_0$. For $G \geq G^*$, the scaffold term becomes $h_{\max}(1 - e^{-(G-G^*)/\tau})$, which is increasing and concave with asymptote $h_{\max}$. Once $c(G)$ has been driven to zero, the net effect is positive and saturating; before that point, subtracting the residual cost only delays the crossing. This yields the observed asymmetric piecewise structure: weak-tie drag is limited, while strong-tie promotion rises quickly and then saturates. $\square$

Theorem 2 (Intervention closure). Under Lemma A and Theorem A, any intervention that length-matches prompts, preserves the task distribution, and increases semantic margins is predicted by the local model to weakly improve both single-step routing and expected downstream rescue, provided it does not reduce a , α , or $Acc(B)$ and higher-order terms do not dominate:

$$\Delta Acc(N) \geq 0, \quad \Delta Acc_{\text{res}}(B) \geq 0, \quad (51)$$

with strict improvement whenever the intervention strictly reduces b and $\alpha(1 - Acc(B)) \ln N > 0$.

Proof. By Lemma A, a length-matched boundary intervention that increases semantic margins can only move the semantic component of the slope, and Proposition A gives $\Delta b \leq 0$. Since $Acc(N) = a - b \ln N + o(\ln N)$, decreasing b weakly increases single-step routing accuracy for $N > 1$. By Theorem A,

$$\frac{\partial Acc_{\text{res}}(B)}{\partial b} = -\alpha(1 - Acc(B)) \ln N. \quad (52)$$

When $\alpha(1 - Acc(B)) \ln N > 0$, a strict decrease in b strictly increases the local model’s expected rescue term. Therefore, under the fixed-headroom and negligible higher-order conditions stated above, the same library-side intervention that improves routing is predicted to improve the expected availability of correct state for downstream execution. $\square$

B. Experimental Details

B.1. Library construction and sampling.

Each routing trial exposes exactly one gold skill and $N-1$ distractors. Distractors are sampled without replacement from the same 14-domain library mixture, with the target domain kept in proportion to the full library unless a local-competition experiment intentionally changes the neighbor set. For multi-step route-only pipelines, all gold skills are fixed by the annotated task graph; only the non-gold exposed skills are resampled. We use fixed sampler seeds for paired comparisons so that baseline and intervention conditions see the same task and distractor instances.

B.2. Task construction and annotation.

Most evaluation tasks are benchmark-derived or adapted from real software-agent task settings. They keep the original task intent and receive additional gold-skill annotations for our library. We use synthetic drafting only for controlled routing cases where existing benchmarks do not require enough distinct skills to support the planned library-size and multi-skill sweeps. These synthetic cases are written from predefined skill combinations, then independently checked against the skill library. A task is retained only when annotators agree on the gold skill or gold skill sequence; disagreements are adjudicated by removing the case rather than relabeling it into the majority class. Synthetic drafting assistance by LLMs (GPT-5.4) is used only before annotation, not for scoring, rewriting, manager optimization, or downstream transfer evaluation.

Human annotation guideline for skills and tasks. Annotators were given the following written guideline.

High-Quality Task Annotation Guideline

Decide whether the task is a valid evaluation item. A valid task must be realistic, executable, and specific enough that another annotator can identify the operation, object, expected artifact, and success condition without seeing model outputs.

Inputs. You will receive a task card with:

- user-visible task instruction;
- benchmark source or synthetic-control flag, when available;
- required output artifact, file, answer, patch, report, or state change;
- pipeline position or candidate composition, when this is a multi-step case;
- any provided files, schema, command context, or environment notes.

You may also inspect the frozen skill-library snapshot to judge whether the task is in scope, but do not choose labels during this pass.

Acceptance criteria. Keep a task only if all criteria below are satisfied.

1. **Clear intent:** the instruction states a concrete user goal, not merely a topic.
2. **Concrete object:** the task identifies or clearly implies the file, repository, API, dataset, table, document, command, state, or other object to operate on.
3. **Explicit operation:** the requested action is identifiable, e.g., analyze, edit, extract, validate, transform, run, compare, summarize, repair, or generate.
4. **Observable output:** the final answer, file, patch, report, state change, or execution artifact can be checked after completion.
5. **Sufficient context:** a competent software agent would not need to invent missing requirements, hidden files, private credentials, unstated schemas, or unprovided external facts.
6. **Library relevance:** at least one available skill is plausibly needed, even if the exact gold skill set is decided later.
7. **No label leakage:** the task should not trivially reveal the answer by copying a skill name or internal label, unless that wording is naturally present in the source benchmark or real user request.
8. **Single coherent request:** the task should not bundle unrelated goals that would naturally be separate tasks.

Permitted light edits. You may make only minimal edits that preserve task intent:

- normalize spelling or formatting;
- expand an abbreviation already clear from context;
- add an explicit expected artifact when it is directly implied by the source;
- split a bundled task only if the source explicitly contains separable subtasks and each subtask can stand alone.

Rejection labels. Use `AMBIGUOUS` when multiple incompatible interpretations are equally reasonable. Use `UNDERSPECIFIED` when key inputs, objects, constraints, or success conditions are missing. Use `OUT_OF_SCOPE` when the task is not an executable software-agent task or no skill-library capability is plausibly relevant.

Reject the task if:

- the requested operation is unclear;



- the expected output cannot be checked;
- key inputs are missing;
- the task requires private credentials, hidden files, or unstated external state;
- it can only be solved by assuming capabilities not present in the task card or library;
- it is primarily open-ended conversation, preference elicitation, or general advice rather than an executable task.

Quality-control rule. Do not inspect model outputs. If annotators disagree on whether a task is ACCEPTED, AMBIGUOUS, UNDERSPECIFIED, or OUT_OF_SCOPE after applying the criteria above, exclude the task rather than forcing agreement by majority vote.

Skill and Pipeline Label Annotation Guideline

Label the minimal sufficient gold skill set or ordered gold-skill sequence using the frozen skill library. The label should contain the skills required to complete the task, not every skill that might be useful.

Inputs. You will receive:

- the accepted task card and expected artifact;
- the frozen skill library, including each skill's name, description, declared inputs, declared outputs, domain tag, examples, and boundary notes when available;
- pipeline metadata when the task is part of a multi-step case.

Core principle: minimal sufficient gold set. A gold skill set is valid only if:

1. **Sufficiency:** together, the selected skills can complete the task under their stated interfaces.
2. **Necessity:** removing any selected skill would leave a required operation, artifact, or dependency uncovered.
3. **Interface faithfulness:** the label must rely on declared skill behavior, not on unstated behavior inferred from a broad name.

Single-step tasks. Assign a gold skill set G .

- Use one skill if a single skill fully covers the task.
- Use multiple skills only when the task genuinely requires multiple capabilities, artifacts, or interfaces and no single available skill covers them.
- Do not add formatting, checking, retrieval, or helper skills unless they are required by the task's success condition.
- Mark AMBIGUOUS if two or more distinct gold sets are equally valid.
- Mark UNDERSPECIFIED if the task lacks information needed to decide the gold set.

Choosing between similar skills. When multiple skills look plausible:

1. Prefer the narrower skill whose description names the concrete object, operation, domain, input, or output in the task.
2. Prefer a skill with matching declared inputs and outputs over a skill that only matches by general wording.
3. Reject broad catch-all skills unless no narrower exposed skill can perform the task.
4. Do not select a skill when the task asks for an excluded behavior, a different artifact type, or a different execution environment.
5. If the difference remains unresolved after checking descriptions, examples, and boundary notes, mark AMBIGUOUS.

Pipeline tasks. Assign an ordered sequence of steps:

Step 1: G_1 , Step 2: G_2 , ..., Step K: G_K .

Each step-level gold set must either:

- produce the final requested artifact;
- produce an intermediate artifact required by a later step; or
- perform a required validation, transformation, or state update named by the task.

Do not add optional helper steps. Do not reorder steps unless the task logic requires that order. If two skills must be used jointly at the same stage, place them in the same step-level gold set rather than creating an artificial sequence.

Dependency labels for adjacent pipeline steps. For each adjacent pair (G_i, G_{i+1}) , assign one label:

tight the downstream step cannot be completed without the upstream artifact or state.



loose the upstream artifact may help, but the downstream step can still be completed from the original task or other provided context.

none the steps are independent controls; the upstream artifact is not needed by the downstream step.

Closure check. For every downstream step, list the required input artifact or state and verify that it is produced by an earlier step or present in the original task card. If any required input has no source, mark the case **UNDERSPECIFIED**.

Submission fields. For each accepted case, submit:

- task id;
- **ACCEPTED** task-quality status;
- gold skill set for single-step tasks, or ordered step-level gold sets for pipeline tasks;
- dependency labels for adjacent pipeline steps;
- one-sentence rationale citing the decisive interface or boundary evidence;
- **AMBIGUOUS** or **UNDERSPECIFIED** flag if no reliable label can be assigned.

Agreement policy. Annotate independently and do not inspect model outputs. A case is kept only when annotators agree on the gold skill set or full ordered sequence and dependency labels. If disagreement remains after reviewing interface match, boundary match, negative evidence, minimality, and pipeline closure, exclude the case rather than resolving by majority vote.

B.3. Routing protocol and parsing.

Most routing experiments use the following plain-text prompt:

You are a skill router. Given a task description and a list of available skills, select the single most appropriate skill.

Task: \{task_description\}

Available skills:

\{skill_list\}

Respond with only the skill name. Do not explain your choice.

We parse the first exact skill-name span after normalizing whitespace and case. If no exposed skill name is returned, the response is counted as hallucination or abstention, not mapped to the nearest valid skill. This is why the paper can distinguish ordinary out-of-library hallucination from in-library hijack. Unless a provider disables temperature control, routing uses deterministic decoding; repeated runs come from independent task/distractor samples or provider-side nondeterminism, not from best-of- k selection.

B.4. Description-quality and anchor manipulations.

Description-quality sweeps hold the task, gold skill, candidate set, and model fixed while changing only the skill-description field. L1 uses names only; higher levels add purpose, inputs/outputs, boundary conditions, and examples. Anchor-loss experiments hold the task intent fixed while removing concrete query anchors such as file names, APIs, schema fields, or domain-specific verbs. Black-hole experiments cross this query manipulation with skill-side abstraction, so capture is measured as an interaction rather than inferred from one degraded condition.

B.5. Execution-state protocol.

No-state execution probes present the two steps jointly but withhold any upstream artifact, thereby estimating interaction before state realization. Correct-state probes execute the upstream skill first and insert the resulting artifact into the downstream routing context. Wrong-state probes condition on incorrect upstream artifacts and

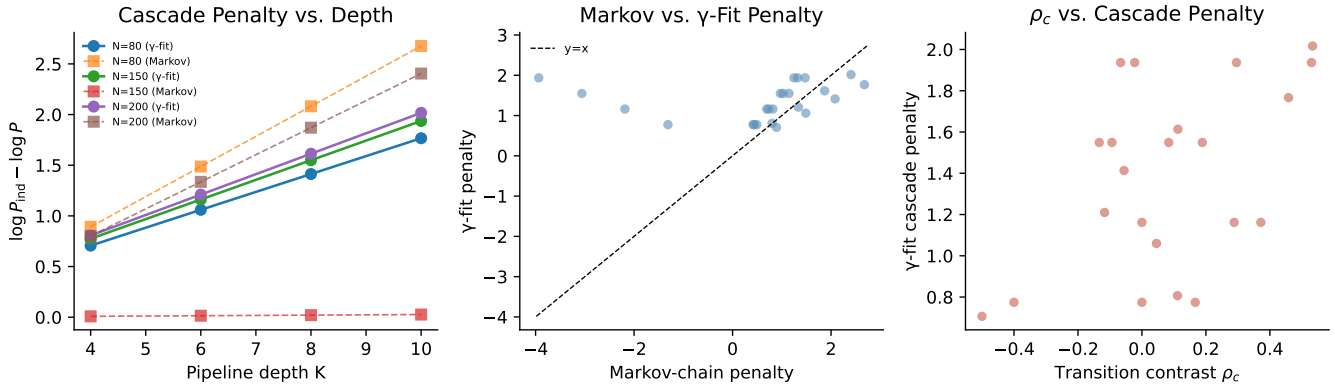


Figure 9: Cascade Penalty Decomposition. Pipeline depth amplifies the gap between observed end-to-end accuracy and the independence baseline.

stratify pairs by dependency strength κ , where tight dependencies require the upstream artifact for the downstream checker and loose dependencies do not. Artifact correctness is scored by task-specific checkers or human verification when a checker is unavailable.

B.6. Fitting and uncertainty.

Logarithmic routing laws are fit by least squares over $N \in \{10, \dots, 500\}$. Binomial rates use Wilson 95% intervals. Cross-model associations use Spearman correlation with percentile bootstrap confidence intervals. Optimization comparisons use paired task sets whenever possible; the headline prospective optimization reports the pre-specified held-out set rather than selecting the best run.

B.7. Routing Cascade Checks

This diagnostic uses route-only multi-step tasks to measure whether pipeline failure is only the product of single-step routing errors or whether later routes become harder after earlier routing decisions have compressed the task state.

Route-only cascade protocol. Each case is an annotated skill sequence $(s_1^*, \dots, s_K^*)$ with the same underlying task graph used across library sizes. At step k , the router sees the current subtask description, the exposed library of size N , and the previous routed skill names, but it does not receive execution artifacts or checker feedback. This isolates planning state from execution state: any downstream degradation comes from routing over a compressed intermediate plan, not from a wrong file, table, API result, or other realized artifact. For every N , the K gold skills are fixed and only the non-gold distractors are resampled under the same domain as the single-step routing sweep (Fig. 9).

Matched product baseline. For each pipeline position k , we run the same subtask as an isolated single-step routing query with the same gold skill and matched distractor sampling. Let $Acc_k(N) = \Pr[\hat{s}_k = s_k^*]$ be this isolated accuracy. The no-cascade product baseline is

$$Acc_{\text{ind}}(N, K) = \prod_{k=1}^K Acc_k(N). \quad (53)$$

We then run the full route-only pipeline on the same task set and record strict success $Acc(N, K) = \Pr[\hat{s}_1 = s_1^*, \dots, \hat{s}_K = s_K^*]$. The extra depth loss is

$$D(N, K) = \log Acc_{\text{ind}}(N, K) - \log Acc(N, K). \quad (54)$$

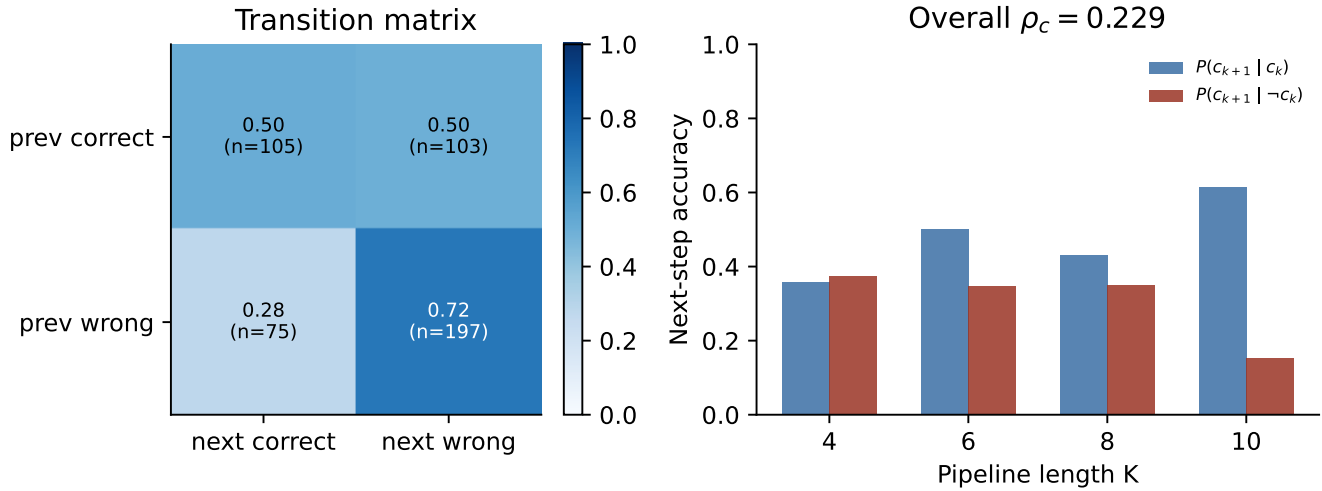


Figure 10: State-Transition Audit. Downstream routing accuracy depends on whether the preceding step remains correct, linking transition state to cascade behavior (annotated by human annotators).

Using log probabilities makes the excess penalty additive across depth and comparable between models with different absolute single-step accuracies.

Transition audit. To check whether the excess loss is consistent with local cascade behavior, we estimate adjacent correctness transitions from the same route-only runs. For every adjacent pair $(k, k + 1)$, define $c_k = 1[\hat{s}_k = s_k^*]$ and estimate

$$\rho_c = \text{Acc}(c_{k+1} | c_k) - \text{Acc}(c_{k+1} | \neg c_k) \quad (55)$$

after pooling adjacent pairs within the same model, library size, and depth condition. A positive ρ_c means the next route is more likely to be correct when the previous route was correct. We compare this transition estimate with the fitted depth exponent γ from the scaling law as a diagnostic, not as an additional fitted result: if both move together, the observed pipeline penalty is attributable to local routing-state compression rather than an unmatched prompt or task-difficulty artifact (Fig. 10).

C. Routing Law

C.1. Empirical Decay Parameters

Table 2 reports the fitted intercept a , decay slope b , and 70% threshold N^* for each evaluated model. Stronger models exhibit smaller b , pushing the operational threshold to larger libraries, but none remove the logarithmic form itself.

Routing-scale experiment details. Each routing condition fixes the evaluated task set and changes only the number of exposed candidate skills. A trial contains the k gold skill and $N - k$ randomly selected distractors sampled without replacement from the domain-stratified library pool. The target distribution is therefore held fixed as N changes: increasing N adds distractors around the same gold tasks rather than introducing harder targets. For paired or repeated settings, the task identifier and candidate-set seed define the unit of aggregation. Repeated calls sharing that unit are averaged first, and task-level means are then averaged across the condition, so tasks with more repeats do not receive extra weight.

Table 2: Fitted parameters for $Acc(N) = a - b \ln N$ across all 15 models. The intercept a can slightly exceed 1 for the strongest models; the law is used over $N \geq 10$.

Model	a	b	R^2	N^* (80%)
GPT-5.4	1.021	0.031	0.981	1246
GPT-5.4-mini	0.998	0.038	0.974	183
Claude Opus 4.6	0.987	0.041	0.979	96
Claude Sonnet 4.6	0.972	0.044	0.976	50
DeepSeek-V4 Pro	0.965	0.046	0.978	36
Gemini 3.1 Pro	0.961	0.047	0.971	31
Qwen3-235B	0.957	0.049	0.973	25
GPT-5-mini	0.953	0.049	0.968	23
Kimi K2.5	0.941	0.052	0.973	15
Kimi K2.6	0.939	0.053	0.971	14
Doubao Seed 2.0 Pro	0.934	0.055	0.969	11
Gemini 3.1 Flash Lite	0.921	0.058	0.972	8
GLM-4.7	0.914	0.060	0.974	7
GLM-5	0.908	0.062	0.975	6
GPT-4o-mini	0.897	0.065	0.971	4

Single-step routing accuracy is estimated for each model and each $N \in \{10, 20, 50, 100, 200, 500\}$ and fit by ordinary least squares as $a - b \ln N$. Pipeline experiments use route-only annotated skill sequences: the gold sequence is known, every step is routed, but no execution artifact from an earlier step is inserted into the next step. This isolates planning and routing from state realization. The independent baseline for a length- K pipeline is computed from the corresponding single-step fit as $(a - b \ln N)^K$. The observed route-only pipeline accuracy is then compared with this baseline to estimate a model-specific compounding exponent γ . Per-step analyses keep the pipeline length fixed while computing accuracy at each step, then use the normalized position k/K only when pooling curves across different lengths.

The fitted intercept a is not used as a probability estimate. It is the intercept at $\ln N = 0$, outside the evaluated range, which is why it can slightly exceed one for the strongest models in Table 2. All claims use the measured sweep $N \in \{10, 20, 50, 100, 200, 500\}$ and the local slope b over that range; the linear-log law is not extrapolated beyond the observed support.

C.2. Decay Slope vs. Semantic Gap Mechanism

This diagnostic connects the fitted routing law to the paper’s main mechanism. If the logarithmic slope b were only a generic model-capacity or prompt-length effect, then it would not depend strongly on where a skill sits inside the library. The local-competition account makes a sharper prediction: skills whose descriptions sit close to a neighbor should lose accuracy faster as more candidates are exposed, because additional skills add plausible near-misses around the same intent. We therefore test whether per-skill decay is largest exactly where the library geometry is most crowded. The visual evidence is Fig. 3: panel (a) shows that sharper descriptions flatten the decay slope, panels (b,c) show where local near-miss errors concentrate, and panel (d) summarizes the same effect through the competition-index fit. This appendix paragraph gives the per-skill regression behind that visual mechanism.

How the diagnostic is run. For each skill, we embed the same description text that appears in the routing prompt and compute the cosine distance to its nearest neighboring skill, denoted $\Delta\mu$. This step uses only the library text and is completed before looking at model outputs. We then estimate a per-skill decay slope using only trials

Table 3: Component attribution for the two highest-leverage library-side optimization actions at $N=150$ (100-task held-out factorial ablation; full details in Appendix E.4).

Boundary Rewrite	Abstract-Skill Removal	Accuracy	Hijack
No	No	71.3%	22.4%
Yes	No	84.1%	9.8%
No	Yes	76.2%	17.3%
Yes	Yes	91.7%	4.1%

where that skill is the gold target across the same library-size sweep as the main routing experiment. Appearances of the skill as a distractor for other targets are used only to define local competition, not to estimate that skill’s own slope.

We correlate the per-skill slope with $1/\Delta\mu$, because a smaller nearest-neighbor gap corresponds to a larger inverse-gap value and hence a predicted larger decay slope. This keeps the diagnostic aligned with the design claim: boundary rewrites should help most when they increase separation between locally confusable skills. In the figure, this is the move from the crowded danger-band regime toward wider local margins; in the regression, it appears as a positive association between inverse gap and decay slope. The relationship is strong: Spearman $\rho=0.71$ ($p<0.001$, $n=452$ skills). A linear regression $b = \beta_0 + \beta_1/\Delta\mu$ gives $\beta_0=0.008$ (95% CI: [0.003, 0.013]), $\beta_1=0.031$ ([0.026, 0.036]), and $R^2=0.49$. The positive intercept means that even isolated skills retain a small baseline decay from finite context and selection noise, but nearly half of the per-skill variation in slope is explained by local semantic crowding. This is why the later optimization targets skill boundaries and nearest-neighbor audits rather than only changing the global prompt.

C.3. Routing Law Controls: Baselines and Task Difficulty

The routing law should not depend on a single implementation of the router, nor should it be an artifact of harder tasks appearing at larger N . This section therefore collects two controls: retrieval baselines under the same scaling sweep, and task-difficulty checks that hold task hardness separate from library size.

C.3.1. Retriever and Router Baselines

Table 4 positions zero-shot LLM routing against sparse and dense retrieval baselines under the same scaling sweep. The purpose is to show that decline with library scale is not unique to one routing implementation.

Note on sample sizes. The baseline table uses $n=50$ tasks per condition, which is intentionally smaller than the $n=500$ used in the main scaling experiments. The baselines are not the primary contribution of the paper; the goal is to check whether sparse retrieval, dense retrieval, and LLM routing all show degradation with larger candidate sets. We report means across runs where repeated runs are available rather than best-of-run values. The low R^2 values for some rows reflect sampling variance at this scale, not absence of the decay trend: at $n=500$, GPT-series achieves $R^2>0.97$ in the main scaling experiment.

BM25 is a lexical control, not the main skill-router regime. The main analysis concerns LLM semantic routing: the model reads natural-language skill descriptions, infers task intent, and can therefore be pulled toward broad semantic attractors. BM25 is included only as a sparse lexical retrieval control. It matches surface terms between the query and skill descriptions, but it does not perform the same semantic abstraction, state-aware skill selection, or execution-conditioned reasoning studied in the paper. This difference is why the large- N reversal is informative. At $N=200$, BM25 (78.0%) outperforms LLM routing (GPT-5.4-mini, 60.7%): BM25 is less exposed to black-hole capture precisely because it lacks the semantic generalization that makes an abstract catch-all skill attractive to an LLM router. The result is therefore not an endorsement of BM25 as a deployment choice; LLM routing is substantially better at small N , handles paraphrase better, and is the relevant regime for skill-based agents.

Table 4: Routing accuracy across methods and library sizes ($n=50$ tasks per condition; means across runs where available). All methods show the same qualitative decline as N grows, but this table is an auxiliary implementation check rather than a headline quantitative fit.

Method	$N=20$	$N=50$	$N=100$	$N=200$	b	R^2
BM25	85.3%	90.0%	79.3%	78.0%	0.041	0.52
Dense Retrieval (BGE)	90.0%	90.7%	88.0%	80.0%	0.041	0.68
LLM (GPT-5.4-mini)	93.3%	92.7%	80.7%	60.7%	0.140	0.82

Rather, the reversal is a negative-control signal: the large-library degradation we analyze is not just generic task difficulty or prompt length, but a semantic in-library hijack mechanism specific to LLM-style skill routing.

C.3.2. Task-Difficulty Confounds

The difficulty concern is that larger libraries might accidentally contain harder tasks, so the observed decay could reflect task selection rather than library size. Our routing sweep is constructed to avoid this: the gold task pool is held fixed. Concretely, every size- N condition draws from the same domain-stratified task pool ($n=500$ per condition in the main sweep); each trial exposes the gold skill plus $N-1$ distractors sampled from the same library mixture. Thus larger N changes the number of alternatives around the same targets, not the target distribution itself.

The control is visible in the fitted scaling curves. Under this fixed-target, distractor-only sweep, all 15 models still follow $Acc(N) = a - b \ln N$ over $N \in \{10, 20, 50, 100, 200, 500\}$, with $R^2=0.968-0.981$ and positive slopes $b=0.031-0.065$ (Table 2). The strongest model has the shallowest decay ($b=0.031$), while GPT-4o-mini has the steepest ($b=0.065$); the sign and log-linear form are unchanged across the model range. Because the task pool does not get harder as N grows, these slopes measure sensitivity to additional distractors rather than a between-condition shift in intrinsic task hardness.

We use the same logic for route-only pipeline controls: the annotated gold skill sequence is fixed by the task graph, and only non-gold exposed skills are resampled. The observed pipeline degradation therefore cannot be explained by longer or harder target tasks being introduced at larger N ; it occurs when the same required skills are routed among more candidate distractors. Finally, per-domain stratification prevents a domain-composition artifact: each size condition preserves the same domain mixture, so the decay is not driven by difficulty becoming overrepresented at large library sizes.

Token-budget boundary. The library-size sweep increases both distractor count and prompt length. Our controls do not make a pure token-budget estimate. They show that the decay is not only generic long-context degradation: the task pool is fixed, length-matched boundary rewrites still move accuracy, and the semantic competition index predicts failures better than N alone. The measured slope therefore applies to the flat exposed-library prompt format studied here, not to pagination, hierarchy, or retrieval-gated top- M exposure.

We also ran a small GPT-5.4-mini context-growth check that keeps the required skill and task pool fixed while increasing only the exposed skill context. The added context contains non-gold skills drawn by the same domain-stratified sampler, so the required skill remains available in every condition and the target tasks do not change. The check is intended as a token-growth stress test, not a replacement for the full 15-model scaling sweep: it asks whether the GPT-5.4-mini decline appears when task hardness is fixed and only the exposed candidate context grows. The observed pattern matches the main sweep direction, supporting the interpretation that scale-induced context growth is part of the flat-library routing problem rather than a separate task-difficulty artifact.

We additionally use a context-matched skill-count control to separate exposed skill count from approximate prompt length. The control uses the same tasks and GPT-5.4-mini router in all conditions. For each task, we first sample the $N=150$ candidate set and use its description-only candidate list to define a fixed token budget. We then

Table 5: Context-matched skill-count control. The same task set and GPT-5.4-mini router are evaluated under approximately matched candidate-list token budgets while changing the number of exposed skills. Candidate budgets are set by the description-only $N=150$ condition, while each evaluated condition renders truncated GPT-5.4-expanded descriptions to match that budget.

Skills	Candidate tokens	Accuracy	Trials
10	5176	60.5%	580
25	5176	56.2%	580
50	5176	51.0%	580
75	5176	54.1%	580
100	5176	53.1%	580
125	5176	53.1%	580
150	5176	49.5%	580

Table 6: Context-scale accounting for the downstream library construction. This is not a pure token-only ablation; it records the scale difference between full exposed context and the gated manager context used in transfer checks.

Context construction	Raw skills	Global cap	Per-task cap	Per-domain cap
Raw collected library	1412	–	–	–
Deduplicated diagnostic library	1141	–	–	–
Manager-gated downstream library	1412	250	80	24

vary the exposed skill count from $N=10$ to 150 while rendering truncated evenly across skills, which utilize GPT-5.4 to expand, to match the same candidate-token budget. Thus the number of selectable in-library alternatives changes while the exposed candidate-list token count remains approximately fixed.

Table 6 makes the context-length boundary explicit. The routing law is estimated on the 1,141-skill diagnostic library after near-duplicate removal, where increasing N also increases the candidate text shown to the model. The downstream manager starts from the broader 1,412-skill raw transfer library and does not claim to hold prompt length fixed; instead, it changes the exposed surface by selecting a smaller domain- and task-conditioned context from that raw library. Therefore the downstream gains should be read as evidence that law-guided gating and editing help under a matched transfer protocol, not as an isolated estimate of the causal effect of tokens.

What each control rules out. The controls separate several possible explanations but do not collapse them into a single causal estimate. The fixed-target sweep rules out task-set hardening across N . Domain stratification rules out a domain-composition shift. Length-matched boundary rewrites rule out a purely token-count account for the boundary intervention, because the candidate list length is matched while local accuracy changes. The per-skill inverse-gap regression and CI fit rule out a pure candidate-count account, because skills with more crowded neighborhoods lose accuracy faster than isolated skills under the same global library size. Finally, the sparse-retrieval control is a negative control for semantic abstraction: BM25 degrades with candidate growth but is less vulnerable to abstract black-hole capture than LLM routing. Together these checks support semantic competition as the dominant measured mechanism in the flat exposed-library protocol, while leaving open how much a hierarchical or retrieval-gated interface would reduce the slope.

C.4. Mechanism Probe: Manipulating Skill Boundaries

The routing law says that larger libraries hurt because they create denser neighborhoods of plausible alternatives. To probe this mechanism directly, we run a *description intervention*: we manipulate the semantic similarity between pairs of skills by rewriting their descriptions to be more or less similar, then test whether routing accuracy changes as predicted by the capacity model.

Local-competition experiment details. The description-quality sweep is paired at the trial level. We hold fixed the task, gold skill, exposed distractor set, model, prompt template, decoding setting, parser, and scoring rule. Only the description field changes. The levels progress from name-only text to brief summaries, target objects and actions, constraints and examples, and finally bounded counterexamples. This design tests whether sharpening the skill boundary changes routing while keeping the underlying task and candidate set unchanged.

Local competition is measured before model inference. For each exposed library, we compute similarity between the gold skill and each exposed non-gold skill, rank the nearest distractors, and count exposure to predefined similarity bands. Error-rate summaries are then grouped by these precomputed ranks and bands. The danger band is selected as the band with the most negative Spearman association between exposure and accuracy: it is the region where adding candidates most reliably hurts. Finally, we compute a competition index

$$CI(s^*, S_N) = \sum_{s_j \in S_N \setminus \{s^*\}} \exp(\beta \text{TFIDFSim}(s^*, s_j)), \quad (56)$$

over exposed non-gold skills and fit the Boltzmann form

$$\text{Acc}(\hat{s} = s^*) = \frac{A}{A + CI(s^*, S_N)}. \quad (57)$$

Here CI is a scalar summary of distractor pressure around the gold skill. The coefficient β is a fitted inverse-temperature parameter: when β is larger, small increases in description similarity create much larger competitive weight. If $\beta = 0$, every exposed distractor contributes equally and CI collapses to a count; positive β makes near neighbors dominate the sum. The parameter $A > 0$ is the fitted gold-side baseline strength, absorbing the target skill’s average salience, description quality, and any model-level preference for the correct class that is not explained by exposed distractors. Thus the fitted probability compares one target mass A against the total distractor mass CI . The gold skill is excluded from CI so that the index measures distractor pressure, not target descriptiveness.

Boundary intervention. We select 60 skill pairs from the danger zone ($\cos \in [0.55, 0.75]$) and 60 pairs from the safe zone ($\cos < 0.55$). For each pair, we produce three description variants: *original*, *pushed apart* (targeting $\cos < 0.45$), and *pulled together* (targeting $\cos > 0.80$). We verify cosine shifts using BGE embeddings before running routing experiments ($n=100$ tasks per pair per variant). All variants are length-matched to within ± 15 tokens and written at L4 quality; functional equivalence is verified by two annotators ($\kappa=0.88$). The manipulation therefore changes the local semantic geometry while holding the skill function fixed.

Mechanism check.

- **Pushed apart:** accuracy increases by +18.4% for danger-zone pairs ($p < 0.001$, $n=60$) and +9.2% for safe-zone pairs ($p=0.003$).
- **Pulled together (danger zone):** accuracy decreases by −11.3% ($p < 0.001$).
- **Pulled together (safe zone):** accuracy *increases* by +3.1% ($p=0.041$), consistent with the near-duplicate disambiguation effect.

All four directional predictions match the capacity model without refitting. This supports the mechanism behind the routing law: description similarity is not merely correlated with routing error, but can causally move probability mass among local competitors.

C.5. External Validity

To validate the routing law beyond the main skill-routing experiments, we test its qualitative logarithmic on ToolBench [5], not as a refit of the main paper’s quantitative slopes (Fig. 11).

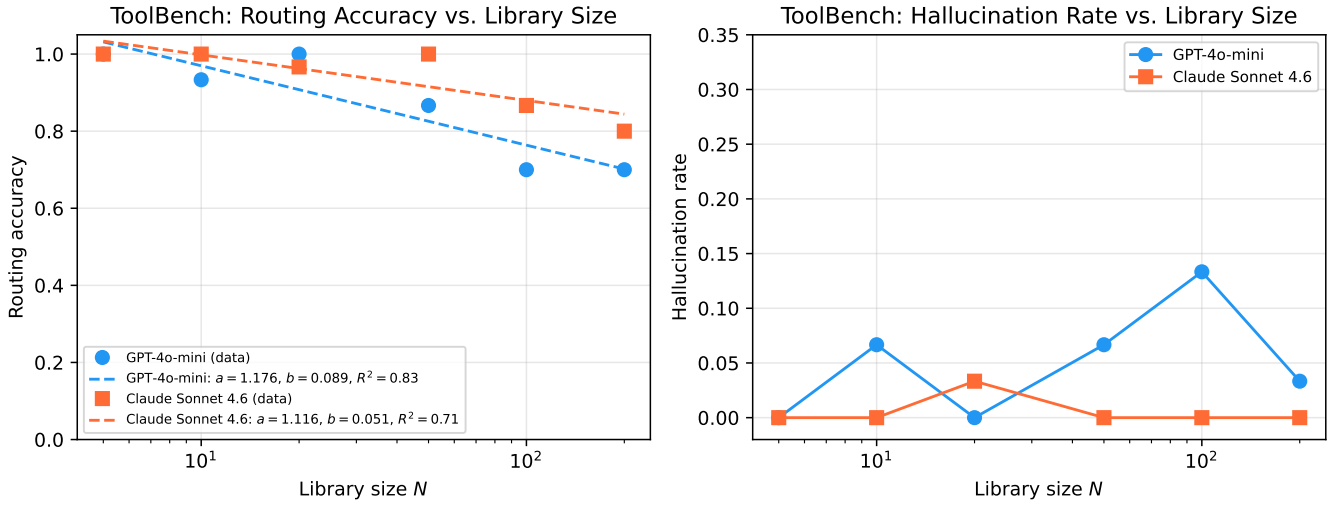


Figure 11: ToolBench Validity. ToolBench routing accuracy and hallucination rates follow the same qualitative library-size pattern on an external tool corpus.

Table 7: ToolBench routing accuracy and hallucination rate by library size. 30 queries per condition, gold API + uniform distractors.

N	GPT-4o-mini		Claude Sonnet 4.6	
	Accuracy	Halluc.	Accuracy	Halluc.
5	1.000	0.000	1.000	0.000
10	0.933	0.067	1.000	0.000
20	1.000	0.000	0.967	0.000
50	0.867	0.067	1.000	0.000
100	0.700	0.133	0.867	0.000
200	0.700	0.033	0.800	0.000
Log fit	$a=1.176, b=0.089, R^2=0.83$		$a=1.116, b=0.051, R^2=0.71$	

External routing setting. We sample 140 answer traces from the ToolBench G1 split and extract 633 unique API functions with their natural-language descriptions. For each of 30 sampled queries per library size $N \in \{5, 10, 20, 50, 100, 200\}$, we construct a library containing the gold API plus $N-1$ distractors drawn uniformly from the full API pool. Both GPT-4o-mini and Claude Sonnet 4.6 route each query to a single API name using the same zero-shot plain-text protocol as the main experiments. The random seed is fixed at 42.

ToolBench measurement details. For each model and library size, each query is scored as correct if the parsed API name matches the gold API from the original ToolBench trace. A response that does not match any exposed API name is counted as hallucination; an exposed but non-gold API is counted as an in-library routing error. Accuracy and hallucination rate are computed over the 30 sampled queries at each N . The log fit is estimated separately for each model from those condition means, using the same $a - b \ln N$ form as the main routing experiment.

Routing-law replication. Table 7 and Fig. 11 show the accuracy and hallucination rates for both models. GPT-4o-mini accuracy declines from 1.000 at $N=5$ to 0.700 at $N=200$; Claude Sonnet 4.6 declines from 1.000 to 0.800. Log fits $Acc(N) = a - b \ln N$ yield: GPT-4o-mini: $a=1.176, b=0.089, R^2=0.83$; Claude Sonnet 4.6: $a=1.116, b=0.051,$

$R^2=0.71$. Both slopes are positive and the qualitative decay pattern replicates across model families. Hallucination rate stays low ($\leq 13\%$) across all N for both models, replicating the main routing result that in-library routing errors dominate over hallucination at scale.

D. Execution Law

D.1. Cross-Model Rescue Robustness

The execution law does not imply that every model receives the same rescue gain. Rescue is largest when the upstream step is likely to be correct and the downstream route has headroom. The cross-model diagnostic therefore serves a narrower role: it checks that the state-rescue pattern is systematic across model families while leaving the precise coefficient fit as diagnostic support rather than a separate headline law.

Execution-state experiment details. The execution-state experiments use ordered two-skill pairs from annotated pipelines. For each pair, we first run matched single-step routing trials for the upstream skill A and the downstream skill B under the same exposed-library regime. These trials estimate $Acc(A)$ and $Acc(B)$ without any execution artifact. The no-state paired condition then asks for the two routes jointly but still withholds upstream output; its observed joint success $Acc(A, B)$ is compared with $Acc(A)Acc(B)$ through $\Delta = Acc(A, B) - Acc(A)Acc(B)$. This is the protocol-leakage control: if pairing alone changes routing, Δ should move away from zero before any state is realized.

How to read the multiplicativity control. The no-state result is a leakage diagnostic, not a theorem of pairwise independence. Small pair-level residuals can occur because the two prompts may share wording or domain context. The key comparison is scale: these no-state residuals are small relative to the downstream changes observed only after a verified upstream artifact is inserted.

For correct-state trials, the upstream skill is actually executed. Only artifacts that pass the upstream checker, or human verification when no checker exists, enter the correct-state condition. The artifact is inserted verbatim into the downstream routing context, while the downstream gold skill and candidate library are kept matched to the no-state condition. Rescue boost is measured as the downstream success gain relative to the matched no-state downstream route, and rescue ratio is measured as the realized joint success divided by the product baseline. The rescue-law regression uses boost rate as the response and rescue potential $(1 - Acc(B))Acc(A)$ as the predictor.

Wrong-state and capability-gap robustness. The wrong-state results are also heterogeneous in a structured way. In the quality propagation audit, 11 models over 68 ordered pairs produce 23,739 downstream-scored rows. Tight dependencies lose quality when the upstream artifact is wrong rather than perfect (-7.2%), while loose dependencies show a small positive difference (+2.8%) because the downstream step can often ignore the bad upstream artifact. Independent pairs stay near zero (-0.8%). The ignored-upstream rate rises in all three groups (about +10.5% to +14.5%), so the loose-pair gain should be read as bad-state rejection or fallback, not as true repair of the wrong artifact.

The capability-gap analysis shows the same kind of structured heterogeneity. Across 11 models and 253 model-pair summaries, large-gap pairs have positive product synergy (+25.2% on average, CI half-width 11.6%), whereas small-gap pairs are near zero (+1.5%, CI half-width 3.4%). The sign is model-stable in the large-gap regime (10 positive models, none negative). Thus the thresholded G result is not used as a universal closed-form deployment rule; it summarizes where positive joint execution is most consistently observed.

The pooled rescue coefficient is reported for the same reason: it tests the state-gated form of the execution law rather than estimating a deployment constant. Different models, dependency types, and domains can change the local magnitude of α , but the predicted structure remains the same: rescue requires correct upstream state and

Table 8: Optimization rules derived from the observed routing and execution regimes. The action names are the terms used consistently in the manager description and ablations.

Failure Regime	Optimization Rule	Grounding Mechanism
Local semantic competition	Nearest-neighbor audit: compare new and existing skills against closest neighbors; merge or rewrite skills in the $[0.55, 0.75)$ danger band.	Local competition: confusion is driven by local crowding, not global library size.
Descriptor ambiguity	Boundary rewrite: add boundary conditions and counterexamples; verify with pair-level routing audits.	Boundary mechanism: boundary clauses are not generic multipliers; reciprocal exclusions reduce off-pair misses.
Anchor loss and black-hole capture	Abstract-skill removal or narrowing: keep broad catch-all skills out of the flat routing pool; preserve concrete operational anchors.	Drift and attractor mechanism: abstract attractors are dangerous only when prompt anchors are weak.
Pipeline fragility	Prompt anchoring: reintroduce user intent at mid-chain steps; order reliable upstream steps before hard downstream choices.	Routing-scale and rescue laws: mid-chain is fragile; correct realized state most strongly helps hard downstream choices.
Execution-state propagation	Runtime context gating and closure checks: prefer loose dependency between steps; when joint execution is needed, pair skills across a sufficient capability gap rather than as weak-tie peers.	Wrong-state and tie-dependent synergy: loose coupling limits error propagation; strong-tie pairs promote weaker steps once $G \geq G^*$, while weak-tie pairs create drag.

downstream headroom, whereas wrong-state effects depend on whether the downstream checker actually needs the upstream artifact.

For wrong-state trials, we deliberately condition on upstream artifacts that fail the upstream checker or receive degraded rubric scores. Pairs are stratified by dependency type from the task graph and by whether the downstream checker requires the upstream artifact. Tight pairs require the upstream output for downstream success; loose pairs keep enough downstream task signal that the artifact can be ignored. Capability-gap trials estimate $G = \text{Acc}(A) - \text{Acc}(B)$ from matched single-step success before joint execution is evaluated. The fitted transition around $G^* \approx 0.25$ is therefore based on a pre-execution capability measure rather than on post-hoc joint-task outcomes.

E. Design Principles for the Automatic Skill Manager

This section explains how the empirical laws are converted into the automatic skill manager evaluated in Sec. 6. It has three roles. First, it fixes the design vocabulary by mapping each observed failure regime to a named library-maintenance action. Second, it specifies the manager implementation: scorecards, planning, edit actions, runtime gating, and closure checks (Appendix E.1). Third, it separates the evidence for the manager into focused checks: boundary-rewrite stability (Appendix E.3), a stage-level factorial ablation (Appendix E.4), and rewrite-style selection (Appendix E.5). The central constraint is that the manager is not treated as a free-form prompt-engineering bundle: each edit is tied to a measured law variable, while the router, task set, candidate construction, prompt template, and parser are held fixed in the held-out routing evaluation.

E.1. Manager Pipeline

Table 8 gives the rulebook used by the manager. Each row starts from a failure regime measured earlier in the paper, turns it into an optimization rule, and names the mechanism that justifies the rule. The table should be read as the design contract for the manager: edits are allowed only when they target a measured source of routing or execution failure.

The automatic skill manager is an analyzer-first library maintenance loop. It does not change the base model

or train a new router. It takes a structured skill library as input, computes law-aligned risks, proposes human-reviewable edits, applies approved changes, and then re-audits the before/after library. The implementation used in the experiments follows six stages: schema normalization, scorecard construction, recommendation planning, edit application, runtime gating, and before/after audit.

Stage 1: schema normalization. Each skill record contains an identifier, name, description, examples, family label, tags, input/output metadata, and explicit anchors: verbs, objects, and constraints. Optional pipeline edges record upstream skill, downstream skill, dependency type (tight, loose, or independent), and edge weight. The manager normalizes YAML/JSON/metadata-derived skills into this schema before scoring.

Stage 2: law-aligned scorecards. The manager computes three scorecard levels. A *pair scorecard* measures lexical or embedding overlap, shared anchors, same-family membership, competition risk, merge candidate score, and weak-drag risk. A *skill scorecard* aggregates each skill’s top-neighbor similarity, family conflict, anchor strength, abstraction score, black-hole risk, routing fragility, and rewrite priority. A *library scorecard* aggregates competition density, danger-zone mass, anchor-weakness mass, black-hole exposure, family-interference index, pipeline-fragility index, and predicted routing stability. These quantities correspond directly to the law variables and action names in Table 8: local competition drives nearest-neighbor audit and boundary rewrite, anchor weakness drives prompt anchoring, black-hole exposure drives abstract-skill removal or narrowing, and pipeline fragility drives runtime context gating and closure checks.

Stage 3: recommendation planning. The planner converts scorecards into pending actions rather than silently modifying the library. High local-competition risk triggers rewrite or review-overlap actions for boundary rewrite. High abstraction or black-hole risk triggers remove, narrow, or rewrite actions for abstract-skill removal or narrowing. Weak anchors trigger prompt-anchoring edits that add concrete verbs, objects, and constraints. Candidate-skill simulation compares a proposed skill to the existing library and returns reject, add-with-review, or approve-candidate based on nearest overlap, same-family hits, anchor strength, abstraction score, and predicted routing stability delta.

Stage 4: library edits. Approved actions are applied as explicit library edits. A boundary rewrite replaces the skill description and anchor fields with narrower verbs, objects, constraints, and reciprocal exclusions. A merge action keeps a target skill, absorbs examples/tags/anchors from dropped skills, and removes redundant entries. A remove or narrow action handles broad catch-all skills that create abstract-attractor risk. After applying a plan, the manager recomputes all scorecards and emits a before/after diff with metric deltas for competition density, danger-zone mass, anchor weakness, black-hole exposure, and predicted routing stability.

Stage 5: runtime context and closure checks. For downstream agent runs, the manager also supports runtime context selection. It scores skills against the task instruction using matches in names, tags, anchors, descriptions, families, and domain profiles, then exposes a domain-gated context rather than the full flat library. Local artifact tasks receive smaller context limits to reduce irrelevant tool/vendor distraction. A closure checker infers required output artifacts from the task or from explicit task IDs, then prevents completion when required files are missing or empty. This implements the execution-law constraint: correct state must be present before downstream decisions or final completion are trusted.

Stage 6: before/after audit. The complete loop is: load the library; score skill, pair, and library risks; plan rewrite/merge/remove actions; apply approved edits; diff before/after scorecards; and evaluate held-out routing plus downstream execution. In the experiments, this loop produced the optimized library evaluated in Sec. 6. The

Table 9: How the appendix evidence supports the automatic skill manager. Each check tests one narrow claim rather than decomposing the entire manager.

Evidence block	Question answered	What it supports
Boundary-rewrite stability	Does reciprocal boundary text move errors away from unrelated skills and back toward the intended local pair?	Boundary rewrites act through localization before they improve exact accuracy.
Factorial ablation	Do two cleanly toggled stages move held-out routing under a fixed router and fixed task set?	Local-competition rewrites and abstract-attractor removal each contribute to the manager’s routing gain.
Rewrite-style selection	Which description format improves geometry diagnostics without adding reusable boilerplate?	The chosen compact rewrite style is an implementation choice inside the boundary stage, not a separate causal result.

implementation is deliberately conservative: every edit is grounded in an observed law variable, and the router is held fixed so gains can be attributed to library geometry rather than model retraining.

E.2. Evidence Map for the Manager

The auto-skill-manager has more than two optimization stages: it audits neighbors, rewrites boundaries, strengthens anchors, merges or reviews overlapping skills, narrows or removes abstract skills, gates runtime context, and checks execution closure. The appendix evidence does not claim to decompose every component independently. Instead, it uses three focused checks that cover the main claims needed for Sec. 6.

Table 9 summarizes the role of each evidence block. Runtime context gating and closure checks are part of the downstream transfer setup; they are not treated as separate routing-law ablation factors. This separation is deliberate. The held-out routing ablation isolates the two largest library-surface actions under a fixed router, prompt template, parser, and task set. The downstream transfer uses the auto-skill-manager because completed benchmark tasks also require runtime context selection and closure checks. We therefore report component evidence for the main routing-law actions and treat runtime execution controls as part of the downstream system configuration, not as additional proof that each manager component has an independent causal effect.

E.3. Boundary-Rewrite Stability Details

Table 10 reports the model-level and pair-level stability checks for the 15-pair boundary-rewrite probe. Each probe pair contains two nearby skills from the danger zone: the gold skill and its most plausible local competitor. The rewrite is *bilateral*: we add reciprocal boundary clauses to both descriptions, so each skill states not only what it does, but also how it differs from the paired neighbor. For example, if two skills both mention repository analysis, the rewritten descriptions explicitly separate the trigger conditions, inputs, and outputs that should send a task to one skill rather than the other.

We track three quantities. *Accuracy gain* is the ordinary change in selecting the gold skill. *Outside-miss change* counts errors that leave the intended two-skill neighborhood entirely: the router selects some third skill, not the gold skill or its paired local competitor. *Local-support change* is the complementary mass assigned to the intended neighborhood, i.e., probability that the router selects either the gold skill or the paired competitor. A positive local-support change does not by itself mean the router is correct; it means the rewrite has moved attention from diffuse unrelated alternatives back to the relevant local decision.

The stable effect across models is therefore *localization*. Boundary text first pulls diffuse outside-pair errors back into the intended two-skill neighborhood. Accuracy then improves only when the localized mass lands on the correct member of the pair rather than the competitor. This is why the table reports both outside-miss reduction and

Table 10: Stability of the bilateral boundary rewrite over 15 prospectively specified pairs. Positive accuracy counts report pairs with boundary accuracy above the original condition; outside-miss and local-support counts report pairs moving in the desired direction.

Analysis unit	Accuracy gain	Outside-miss change	Local-support change
GPT-4o-mini	+11.3%	−19.7%	+19.7%
GPT-5.4-mini	+9.3%	−9.7%	+9.7%
Pooled pairs	+10.3%	−14.7%	+14.7%

Table 11: Stage-level 2×2 ablation at $N=150$ on a 100-task held-out set. “Boundary Rewrite” probes the local-competition stage after nearest-neighbor audit; “Abstract-Skill Removal” probes the anchor-loss/black-hole stage. The auto-skill-manager also includes prompt anchoring, overlap review, runtime context gating, and closure checks.

Boundary Rewrite	Abstract-Skill Removal	Accuracy	Hijack
No	No	71.3%	22.4%
Yes	No	84.1%	9.8%
No	Yes	76.2%	17.3%
Yes	Yes	91.7%	4.1%

accuracy gain: the first measures whether the rewrite restored the right neighborhood, and the second measures whether it also resolved the within-pair boundary.

E.4. Design Evidence: Factorial Ablation of Bundled Optimization

The prospective evaluation (Sec. 6) reports a bundled 20.4% gain from the optimized library. The ablation below is a stage-level mechanism probe, not a complete decomposition of every manager component. It toggles one routing-law stage and one attractor-law stage. *Boundary rewrite* is the ablated form of the nearest-neighbor-audit/boundary-rewrite action in Table 8: crowded pairs are identified first, and rewriting then adds boundary clauses, concrete anchors, and scope exclusions. *Abstract-skill removal* represents the anchor-loss/black-hole action: broad catch-all skills are removed from the flat routing pool or narrowed so that they no longer absorb vague prompts. We run the 2×2 factorial ablation on a 100-task held-out set at $N=150$, keeping the router, task set, distractor construction, prompt template, and parser fixed.

Boundary rewrite alone accounts for 12.8% of the accuracy gain ($84.1 - 71.3$) and 12.6% of the hijack reduction. Abstract-skill removal alone contributes 4.9% accuracy ($76.2 - 71.3$) and 5.1% hijack reduction. The two effects sum to 17.7%; the remaining 2.7% ($91.7 - 71.3 - 12.8 - 4.9$) is the interaction, consistent with the dual-trigger mechanism: removing abstract skills is most effective when descriptions are already concrete enough to prevent vague-prompt drift. Thus the ablation validates two representative law channels used by the manager: local pairwise confusion and abstract-attractor capture. It should be read as a mechanism probe for two stages of the optimization pipeline, not as an exhaustive list of all manager actions.

E.5. Design Evidence: Rewrite Style Selection

The boundary-rewrite stage in the ablation still leaves an implementation choice: how should rewritten skill descriptions be expressed? This check is an implementation choice inside the boundary-rewrite stage, not a separate causal claim. Table 12 compares five rewrite styles by the same library-geometry quantities used in the routing law: stability, anchor weakness, danger-zone mass, and black-hole exposure. Longer structured cards can improve a manager-native routing check while adding shared boilerplate that leaves competition density or danger-zone mass less improved. The final label-free compact card keeps the useful identity, action, and boundary information while minimizing reusable template language.

Table 12: Rewrite-style selection using library-geometry diagnostics. Stability is better when higher; anchor weakness, danger-zone mass, and black-hole exposure are better when lower. Manager-native routing accuracy is used only to select the rewrite style.

Rewrite form	Actions	Δ Stab.	Δ Anchor	Δ Danger	Δ Black-hole	Δ Comp.	Mgr. Acc.	Reg.
Pair boundary rewrite	72	+0.020	-0.048	-0.001	-0.017	-0.001	0.811→1.000	0/180
Long structured card	60	+0.003	-0.039	+0.008	-0.011	+0.029	0.778→1.000	0/180
Compact labeled card	60	+0.012	-0.040	-0.003	-0.014	+0.010	0.783→0.994	0/180
Low-boilerplate labeled card	60	+0.017	-0.040	-0.008	-0.015	+0.000	0.783→0.994	0/180
Label-free compact card	60	+0.018	-0.040	-0.009	-0.017	-0.003	0.783→0.994	0/180

Table 13: Task-domain coverage of the downstream transfer data.

Suite	Tasks	Domains	Largest domains
CLAWBENCH	242	18	document editing (18), email (18), cross-domain (17), data analysis (17), workflow automation (17), plus 13 smaller domains
CLAWMARK	100	13	research assistant (15), content operation (12), HR (11), e-commerce (9), journalist (8), product management (8), plus 7 smaller domains

This analysis only selects the rewrite style used inside the boundary-rewrite stage of the held-out optimization. The causal claim remains the factorial result in Appendix E.4 and the prospective held-out evaluation in Sec. 6.

F. Downstream Transfer: CLAWBENCH & CLAWMARK

This section reports the downstream skill-based agent checks referenced in Sec. 6. These runs test whether the law-guided library changes transfer beyond diagnostic routing tasks. They are not used to establish the routing or execution laws, which are measured in controlled probes.

The downstream suites are intentionally broader than the controlled routing probes: CLAWBENCH covers 242 task entries across 18 tool-oriented domains, while CLAWMARK covers 100 held-out tasks across 13 role-oriented domains (Table 13). This coverage table describes the transfer data only; the routing laws are fitted on the separate controlled routing sweeps where task identity and domain mixture are held fixed across library sizes.

Setup. Using gpt-5.4-mini as the base model, we compare raw and optimized skill-library configurations under matched downstream execution settings. The auto-skill-manager uses the same law-guided changes as the prospective routing optimization: nearest-neighbor audit, boundary rewrite, abstract-skill removal or narrowing, and prompt anchoring, etc. The downstream comparison holds the base model, execution prompt family, task split, and scoring rule fixed between raw and optimized libraries. Mean pass rate is computed as the domain-macro average from the benchmark checkers within each downstream suite and is reported separately for CLAWBENCH and CLAWMARK.

Transfer measurement details. Held-out routing transfer is measured on the pre-specified 1,600-task set at $N=150$. The raw-library and optimized-library conditions use the same model, prompt template, parser, task list, candidate-set construction, and scoring rule. The optimized condition changes only the library surface produced by the law-guided manager: boundary rewrites, nearest-neighbor audit, abstract-skill removal or narrowing, and prompt anchoring. We report two routing outcomes. Accuracy is the fraction of tasks routed to the gold skill. Hijack rate is the fraction routed to an exposed but non-gold skill, so hallucinations are not mixed into the in-library failure rate.

Downstream transfer uses matched raw-library and auto-skill-manager-library runs under the same execution setting. This separation matters because downstream success depends both on the library surface and on the

Table 14: Domain-level downstream transfer on CLAWMARK. Entries are pass@4 rates; the Avg. row reports the domain-macro average.

Setting	Tasks	Raw	Auto-Skill-Manager
Clinical Ass.	4	37.5	43.8
Content Ope.	12	22.9	29.2
E-Com.	9	22.2	30.6
EDA	1	25.0	25.0
Exe. Ass.	7	28.6	35.7
HR	11	36.4	43.2
Insurance	7	32.1	39.3
Inv Ana.	6	29.2	37.5
Jour.	8	28.1	34.4
Legal Ass.	6	20.8	25.0
Proj. Man.	8	18.8	21.9
Real Est.	6	37.5	45.8
Research Ass.	15	30.0	36.7
Avg.	–	28.4	34.5

execution policy that preserves or loses intermediate state. We therefore treat the downstream result as a transfer check for the optimized library, not as evidence used to fit the routing or execution laws. The comparison is therefore deliberately narrower than a full agent-system ablation: it asks whether the same library-side changes that improve controlled routing remain useful when benchmark checkers score completed tasks. It does not isolate every runtime component, and it should not be read as evidence that library editing alone solves execution-state propagation.

Result and interpretation. The optimized library improves downstream mean pass rate from 49.3% to 61.6% on CLAWBENCH and from 28.4% to 34.5% on CLAWMARK. This pattern is useful but bounded: the library changes improve downstream transfer, while downstream success still depends on execution-state management. This is consistent with the paper’s two-law picture: library structure controls routing-side probability mass, and execution-state management controls whether downstream state is usable.

Table 15: Domain-level downstream transfer on CLAWBENCH. Entries are pass@4 rates; the Avg. row reports the domain-macro average.

Setting	Tasks	Raw	Auto-Skill-Manager
Calendar	15	91.7	93.3
Code Ass.	15	80.0	86.7
Comm.	15	80.0	86.7
Cross Dom.	17	33.8	41.2
Data Ana.	17	77.9	88.2
Database	5	20.0	40.0
Debug.	5	0.0	20.0
Doc. Edit.	18	51.4	73.6
Email	18	48.6	55.6
File Ope.	15	65.0	86.7
Memory	15	58.3	75.0
Multimodal	15	56.7	81.7
Plan.	5	10.0	20.0
Real Tools	5	15.0	20.0
Security	15	56.7	61.7
System Admin	15	43.3	61.7
Web Brows.	15	41.7	56.7
Workflow Auto.	17	57.4	60.3
Avg.	–	49.3	61.6



Author Contributions

Authors are listed in alphabetical order.

Core Contributors. Charles Chen, Qiming Yu

Contributors. Yuhang Gu, Zhuoye Huang, Hanjing Li, Hongyu Liu, Jinhao Liu, Simin Liu, Dengyun Peng, Jiangyi Wang, Zheng Yan

Supervisors. Carl Che, Mengkang Hu, Fanqing Meng, Ethan Qin